Angular Learning Series

Phase 4 — Routing & Navigation
Phase 5 — Data & Forms

- Routing Fundamentals
- Route Guards
- Lazy Loading
- Nested Routes & Layout Components
- HttpClient & API Integration
- HTTP Interceptors
- Reactive Forms
- Form Validators
- Template-Driven Forms
- Error Handling in Angular

Generated: June 23, 2026

ROUTING FUNDAMENTALS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The Router API is large and stateful; understanding how URL changes drive component lifecycle (rather than the other way around) requires a mental model shift. Understanding navigation lifecycle, parameter reactivity \u2014 especially when navigating between routes that reuse the same component \u2014 and the relationship between the Router's Observable-based APIs and Signals requires real depth.

Angular Relevance: Critical \u2014 Every multi-page Angular app depends on the Router. It controls what renders, what data loads, what guards run, and how deep links work in production. There is no professional Angular app without it.

Section 1 \u2014 Explanation

The GPS Navigator Analogy

Think of your Angular app as a city, and each component as a building. The Router is the GPS navigation system sitting in the user's car.

You don't manually drive the user from building to building \u2014 you give the GPS a destination (a URL), and it figures out the route: which roads to take (which components to render), what to do at each junction (route configuration), and what information to carry along the way (route parameters, like a delivery address baked into the destination).

Crucially, a GPS doesn't tear down and rebuild the car every time you arrive somewhere new. If you're already driving and just change the destination slightly \u2014 say, from "123 Main St" to "125 Main St" \u2014 the GPS recalculates the route, but the car (the component instance) might stay exactly the same if Angular decides the same component handles both addresses. This is the subtle, frequently-misunderstood part of routing: navigating to a new route doesn't always mean a new component instance is created.

Just as a GPS doesn't physically move the car but instructs what path to take, the Router doesn't create or destroy components directly \u2014 it matches the URL to a configuration and lets Angular's component machinery handle the rest.

Technical Explanation

Angular Router is a URL-to-component mapping engine with side effects. When the URL changes (user click, back button, or code call), the Router parses the new URL into a route tree, matches segments against the route configuration, runs any applicable guards, resolves any resolvers, activates the matched component(s) inside `<router-outlet>`, and updates `ActivatedRoute` with current params, query params, and data. The Router is location-aware \u2014 it integrates with the browser's History API so the back button works naturally.

Router Core Concepts

Concept	What It Is	Where It Lives
Routes array	Config mapping URL patterns to components	app.routes.ts
`<router-outlet>`	Placeholder where matched component renders	Template
RouterLink	Directive for declarative navigation	Template
Router service	Programmatic navigation and URL queries	Injected service
ActivatedRoute	Current route's params, query params, data	Injected service
RouterLinkActive	Adds CSS class when route is active	Template
Route params (`:id`)	Dynamic URL segments	URL path
Query params (`?q=`)	Optional key-value pairs after `?`	URL query string
`provideRouter()`	Registers router in standalone app	app.config.ts

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular uses standalone routing \u2014 no `RouterModule.forRoot()` wrapping needed. Routes are provided via `provideRouter(routes)` in `app.config.ts`. Router and ActivatedRoute are injected via `inject()` \u2014 never constructor injection.

Three ways to read route parameters \u2014 oldest to newest:

Approach 1 \u2014 `snapshot.paramMap` (one-time read, not reactive)

Use only when the component is guaranteed to be destroyed and recreated on every navigation. Never use for master-detail or sibling-route navigation.

Approach 2 \u2014 `paramMap` Observable (reactive, classic RxJS)

The reliable fallback. Reacts to every param change regardless of whether the component instance is reused. Pair with `switchMap` for HTTP calls and `takeUntilDestroyed()` for cleanup.

Approach 3 \u2014 `input()` + `withComponentInputBinding()` (Angular v16+, modern standard)

Route params are bound directly as component inputs \u2014 no `ActivatedRoute` injection needed at all. Requires `withComponentInputBinding()` to be explicitly added to `provideRouter()`. The input property name must match the route param name exactly (`:id` \u2192 `id = input.required<string>()`).

Migration path from legacy:

Legacy Pattern	Modern Replacement
`RouterModule.forRoot(routes)`	`provideRouter(routes)` in app.config.ts
`RouterModule.forChild(routes)`	`provideRouter(routes)` or route-level `loadChildren`
Constructor injection of `Router`	`private router = inject(Router)`
Constructor injection of `ActivatedRoute`	`private route = inject(ActivatedRoute)`
`snapshot.paramMap` in `ngOnInit`	`paramMap` Observable or `input()` + `withComponentInputBinding()`
`@Input()` for route param binding	`input()` signal with `withComponentInputBinding()`

Section 3 \u2014 Code Example

Plain TS \u2014 route matching is just a lookup table

```
// Routes is just an array \u2014 no magic, no decorators
const routes = [
  { path: 'home',        component: 'HomeComponent' },
  { path: 'users/:id',   component: 'UserDetailComponent' }, // :id is a param slot
  { path: '',            redirectTo: 'home', pathMatch: 'full' },
  { path: '**',          component: 'NotFoundComponent' },   // wildcard \u2014 must be last
];

// When URL = '/users/42'
// Router finds 'users/:id', extracts { id: '42' } \u2014 always a string, never a number
```

app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { UserListComponent } from './users/user-list.component';
import { UserDetailComponent } from './users/user-detail.component';
import { NotFoundComponent } from './not-found.component';

export const APP_ROUTES: Routes = [
  // Redirect empty path to /home \u2014 pathMatch: 'full' is required here
  // Without it, '' matches every URL by prefix and all routes redirect
  { path: '', redirectTo: 'home', pathMatch: 'full' },

  { path: 'home', component: HomeComponent },
  { path: 'users', component: UserListComponent },

  // Dynamic route \u2014 :id is a required URL segment, always resolved as string
  { path: 'users/:id', component: UserDetailComponent },

  // Wildcard \u2014 MUST be the last entry, catches all unmatched URLs
  { path: '**', component: NotFoundComponent },
];
```

app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { APP_ROUTES } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    // withComponentInputBinding() enables Approach 3 \u2014 input()-bound route params
    provideRouter(APP_ROUTES, withComponentInputBinding()),
  ],
};
```

app.component.ts \u2014 the shell

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  // RouterOutlet, RouterLink, RouterLinkActive must be explicitly imported
  // in standalone components \u2014 they are not globally available
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <!-- RouterLink prevents full page reload; uses HTML5 pushState -->
      <a routerLink="/home" routerLinkActive="active-link">Home</a>
      <a routerLink="/users" routerLinkActive="active-link">Users</a>
    </nav>
    <!-- RouterOutlet is the render target; matched component renders here -->
    <router-outlet />
  `,
})
export class AppComponent {}
```

user-detail.component.ts \u2014 Approach 1: snapshot (one-time, fragile)

```
export class UserDetailComponent {
  private route = inject(ActivatedRoute);

  // \u26a0\ufe0f Read ONCE when component is created \u2014 never updates on sibling navigation
  // Safe only when component is guaranteed destroyed/recreated on every navigation
  userId = this.route.snapshot.paramMap.get('id');
}
```

user-detail.component.ts \u2014 Approach 2: paramMap Observable (reactive, reliable)

```
import { Component, inject, signal } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { switchMap, map } from 'rxjs/operators';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class UserDetailComponent {
  private route       = inject(ActivatedRoute);
  private userService = inject(UserService);

  user = signal<User | null>(null);

  constructor() {
```

```
    this.route.paramMap.pipe(
      map(params => params.get('id')!),
      switchMap(id => this.userService.getUserById(+id)), // + converts string to number
      takeUntilDestroyed()  // auto-cleans when component is destroyed (Topic 27)
    ).subscribe(user => this.user.set(user));
  }
}
```

user-detail.component.ts \u2014 Approach 3: input() + withComponentInputBinding() (modern standard)

```
import { Component, inject } from '@angular/core';
import { input } from '@angular/core';
import { toSignal, toObservable } from '@angular/core/rxjs-interop';
import { switchMap } from 'rxjs/operators';

export class UserDetailComponent {
  // Route param ':id' is automatically bound to this input \u2014 matched by name
  // Requires withComponentInputBinding() in provideRouter() \u2014 see app.config.ts
  id = input.required<string>();

  private userService = inject(UserService);

  // toObservable converts the signal to a stream; switchMap re-fetches on every id change
  user = toSignal(
    toObservable(this.id).pipe(
      switchMap(id => this.userService.getUserById(+id))
    )
  );
}
```

user-list.component.ts \u2014 programmatic and declarative navigation

```
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [RouterLink],
  template: `
    <ul>
      @for (user of users; track user.id) {
        <!-- Array syntax: Router handles encoding of segments -->
        <li><a [routerLink]="['/users', user.id]">{{ user.name }}</a></li>
      }
    </ul>
  `,
})
export class UserListComponent {
  private router = inject(Router);

  users = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
  ];

  openUser(id: number): void {
```

```
    this.router.navigate(['/users', id]);
  }

  openUserWithQueryParams(id: number): void {
    this.router.navigate(['/users', id], {
      queryParams: { tab: 'orders', sort: 'recent' }
      // Result: /users/42?tab=orders&sort=recent
    });
  }
}
```

Query params \u2014 reactive source of truth

```
// Writing query params \u2014 preserving existing ones
this.router.navigate([], {
  relativeTo: this.route,
  queryParams: { category: 'electronics' },
  queryParamsHandling: 'merge', // keeps other params already in the URL
});

// Reading query params reactively
export class ProductListComponent {
  private route          = inject(ActivatedRoute);
  private router         = inject(Router);
  private productService = inject(ProductService);

  products = toSignal(
    this.route.queryParamMap.pipe(
      map(params => params.get('category') ?? 'all'),
      switchMap(category => this.productService.getByCategory(category))
    ),
    { initialValue: [] }
  );

  updateCategory(event: Event): void {
    const category = (event.target as HTMLSelectElement).value;
    // Writing to the URL triggers queryParamMap above automatically
    this.router.navigate([], {
      relativeTo: this.route,
      queryParams: { category },
      queryParamsHandling: 'merge',
    });
  }
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 snapshot in a component that gets reused

```
export class UserDetailComponent implements OnInit {
  private route       = inject(ActivatedRoute);
  private userService = inject(UserService);

  user: User | null = null;

  ngOnInit(): void {
```

```
    // snapshot.paramMap is a one-time read taken at the moment the component mounts
    const id = this.route.snapshot.paramMap.get('id')!;
    this.userService.getUserById(+id).subscribe(u => this.user = u);
  }
}
```

What breaks: If the user navigates from `/users/1` to `/users/2`, Angular reuses the same `UserDetailComponent` instance \u2014 no destroy/recreate cycle occurs. `ngOnInit` does not fire again. The snapshot stays frozen at `id: '1'`. The page silently shows the wrong user's data with no error and no console warning.

Why Angular reuses: recreating the entire component tree on every navigation would be wasteful, especially for components embedded in shared layouts (sidebars, headers). Reuse is the default and correct behaviour \u2014 the bug is reading params incorrectly, not the reuse itself.

\u2705 After \u2014 reactive paramMap subscription

```
export class UserDetailComponent {
  private route       = inject(ActivatedRoute);
  private userService = inject(UserService);

  user = signal<User | null>(null);

  constructor() {
    this.route.paramMap.pipe(
      map(params => params.get('id')!),
      switchMap(id => this.userService.getUserById(+id)), // cancels previous if id changes fast
      takeUntilDestroyed()
    ).subscribe(user => this.user.set(user));
  }
}
```

Why this matters: `paramMap` is a live Observable that emits on every URL param change, regardless of whether the component instance was reused or recreated. `switchMap` automatically cancels any in-flight HTTP request if the user navigates again before the previous one completes \u2014 preventing race conditions.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Using snapshot for components that handle multiple records

What it is: `this.route.snapshot.paramMap.get('id')` inside `ngOnInit()` in a component navigated to with different param values.

Why it happens: Works perfectly on first load and in isolated testing \u2014 the bug only surfaces when navigating between sibling routes (e.g. `/users/1` \u2192 `/users/2`) without going through a different route type first, which would force a component recreate.

How to avoid: Default to `paramMap` Observable or `input()` + `withComponentInputBinding()`. Use `snapshot` only when you can prove the component is always freshly created \u2014 e.g. it's only

ever navigated to from a completely different component type.

Angular consequence: Stale data displayed silently \u2014 no error, no warning, just the wrong record shown to the user after navigating.

Mistake 2 \u2014 Forgetting `withComponentInputBinding()` and wondering why `input()` doesn't populate

What it is:

```
export class UserDetailComponent {
  id = input.required<string>(); // never receives a value \u2014 stays permanently empty
}
```

Why it happens: The `input()`-bound route param pattern is newer, and the registration step in `app.config.ts` is invisible at the component level. The failure is silent until runtime.

How to avoid: Always pair `input()`-based route param binding with `withComponentInputBinding()` in `provideRouter()`. And ensure the input property name matches the route param name exactly \u2014 `:id` in the path must correspond to `id = input.required<string>()`, not `userId` or any other name.

Angular consequence: `input.required<string>()` throws at runtime \u2014 `NG0950: Input is required but no value is available yet` \u2014 because Angular never satisfies the required input.

Mistake 3 \u2014 Assuming route params are numbers

What it is: `this.userService.getUser(params.get('id'))` where `getUser` expects a `number`.

Why it happens: URLs like `/users/42` make `42` feel like a number. It looks like a number, the route is named with a number, so developers pass it directly.

How to avoid: Route params are always `string | null` from the Router \u2014 always convert before use: `Number(params.get('id'))` or `+params.get('id')!`. Add TypeScript interfaces that enforce the typed conversion at the service boundary.

Angular consequence: API calls pass the string `"42"` instead of the number `42` \u2014 may silently work on lenient backends, fails on strictly typed ones with a runtime error that has nothing obvious to do with routing.

Mistake 4 \u2014 Placing the wildcard route before other routes

What it is: `{ path: '**', component: NotFoundComponent }` placed anywhere except the last entry in the routes array.

Why it happens: Order doesn't feel significant in a declarative configuration object. Developers don't think of route arrays as imperative ordered logic.

How to avoid: Angular's Router matches routes top-to-bottom, first-match-wins. The wildcard `**` matches every URL. Any route defined after it is permanently unreachable.

Angular consequence: All routes below the wildcard silently render `NotFoundComponent` instead \u2014 no error thrown, no log entry, just the 404 page appearing for valid URLs.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T	
Use `paramMap` Observable or `input()` + `withComponentInputBinding()` for reactive params	Use `snapshot.paramMap` in any component that can receive different params while active	
Always `Number()` or `+` convert route params before passing to services	Assume params are typed \u2014 they are always `string \	null` from the Router
Use `provideRouter(routes)` in `app.config.ts` for standalone route registration	Use `RouterModule.forRoot(routes)` \u2014 legacy NgModule pattern	
Use `inject(Router)` and `inject(ActivatedRoute)`	Constructor-inject Router or ActivatedRoute	
Import `RouterLink`, `RouterLinkActive`, `RouterOutlet` explicitly in every standalone component that uses them	Forget to import Router directives \u2014 the template compiles but links do nothing	
Use `[routerLink]="['/users', id]"` (array syntax) for dynamic segments	Build URL strings manually: `routerLink="'/users/' + id"` \u2014 brittle, no encoding	
Place `{ path: '**' }` as the absolute last route	Place wildcard before any real routes \u2014 it captures everything below it	
Use `queryParamsHandling: 'merge'` when updating only one query param	Overwrite all query params when only one needed to change	
Use relative navigation inside nested feature routes	Hardcode absolute paths inside deeply nested components	
Reset local component state inside the `paramMap` pipeline with `tap()` when the component is reused	Assume Angular clears local component state on navigation \u2014 it does not	

Section 7 \u2014 Interview Prep

Question 1

"Does Angular always create a new component instance when you navigate to a new route?"

Strong answer covers:

- No \u2014 Angular reuses the component instance when the same component is matched by both the old and new route, for performance reasons
- This means `ngOnInit` does not fire again on such navigations
- Route parameters must be read reactively (`paramMap` Observable or `input()` binding) to respond correctly to this kind of navigation
- `snapshot.paramMap` is only safe when the component is guaranteed to be destroyed and recreated
- Any non-route-derived local state persists across reused-instance navigations unless explicitly reset inside the `paramMap` pipeline using `tap()`

Weak answer misses: That `ngOnInit` doesn't re-fire \u2014 many candidates assume routing always means a fresh component, leading to subtle production bugs they've never correctly diagnosed.

Level: Mid

Question 2

"What is `withComponentInputBinding()` and what problem does it solve?"

Strong answer covers:

- A `provideRouter()` feature (Angular v16+) that automatically binds route parameters, query parameters, and route data directly to component `input()` properties by matching names
- Eliminates the need to manually inject `ActivatedRoute` and subscribe to `paramMap` for simple cases
- Input property name must match the route parameter name exactly
- Without it, `input()`-bound route params silently stay empty, surfacing as `NG0950` errors for required inputs
- Reduces boilerplate significantly for components that only need the param value, not full `ActivatedRoute` access

Weak answer misses: That this feature must be explicitly enabled \u2014 many candidates familiar with the concept assume it's the router default.

Level: Mid\u2013Senior

Question 3

"Explain `pathMatch: 'full'` \u2014 why does the empty path redirect break without it?"

Strong answer covers:

- By default, Angular Router matches routes by prefix \u2014 `path: ''` matches every URL because every URL starts with an empty string
- `pathMatch: 'full'` means the entire remaining URL must equal the path exactly
- Without it on `{ path: '', redirectTo: 'home' }`, the redirect fires for all routes including `/users/42`
- `pathMatch: 'prefix'` (default) is appropriate for most routes; `'full'` is required specifically for empty-path redirects
- This is one of the most frequent "my app redirects everything to home" bugs

Weak answer misses: Explaining *why* prefix matching causes all routes to match the empty path \u2014 candidates who just say "you need it" without explaining the mechanism aren't demonstrating real understanding.

Level: Junior\u2013Mid

Section 8 \u2014 Quick Reference Card

3-line concept definition:

The Angular Router maps browser URLs to components via a plain `Routes` array. `<router-outlet>` is where the matched component renders. Angular reuses component instances by default when the same component handles multiple routes \u2014 so route parameters must always be read reactively.

Syntax at a glance:

Task	API
Register routes (standalone)	`provideRouter(routes)` in app.config.ts
Enable input-bound route params	`provideRouter(routes, withComponentInputBinding())`
Navigate in template	`[routerLink]="['/users', id]"`
Navigate in code	`router.navigate(['/users', id])`
Navigate relative to current	`router.navigate(['../'], { relativeTo: this.route })`
Read path param \u2014 once (fragile)	`route.snapshot.paramMap.get('id')`
Read path param \u2014 reactive (Observable)	`route.paramMap.pipe(map(p => p.get('id')))`
Read path param \u2014 reactive (Signal)	`id = input.required<string>()` + `withComponentInputBinding()`

Task	API
Read query param reactively	`route.queryParamMap.pipe(map(p => p.get('key')))`
Navigate with query params	`router.navigate(['/path'], { queryParams: { key: val } })`
Merge query params without replacing	`{ queryParamsHandling: 'merge' }`
Active link CSS class	`routerLinkActive="css-class"`

When to use / when not to use:

Use `snapshot.paramMap` only when the component is navigated to from a completely different component type \u2014 guaranteed fresh instance every time. Use `paramMap` Observable or `input()` for everything else, which is most of the time.

Use `withComponentInputBinding()` when the component only needs the param value and nothing else from `ActivatedRoute`. Use `paramMap` Observable when you need full `ActivatedRoute` context (query params, data, fragment) or need to combine the param with another reactive source via `combineLatest`.

Choosing how to read route params:

Situation	Use
Component is guaranteed destroyed/recreated on every navigation	`snapshot.paramMap`
Component may be reused; need full ActivatedRoute context	`paramMap` Observable
Component only needs the route param value itself	`input()` + `withComponentInputBinding()`
Need to combine route param with another reactive source	`paramMap` Observable + `combineLatest` / `switchMap`

The one rule that matters most:

Never assume a fresh component instance on navigation \u2014 always read route parameters reactively, because Angular reuses component instances by default whenever the same component matches the new route.

ROUTE GUARDS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 Guards are conceptually simple (return true/false), but the real complexity lies in composing them correctly with async data, redirect logic, and the Resolve pattern which requires understanding the full navigation lifecycle.

Angular Relevance: Critical \u2014 Every production Angular app uses guards. Authentication walls, unsaved-changes protection, and pre-fetched data are all guard responsibilities. Getting them wrong silently exposes protected routes or breaks navigation flow.

Section 1 \u2014 Explanation

The Nightclub Bouncer Analogy

Imagine every route in your app is a room in a nightclub.

`CanActivate` is the bouncer at the door \u2014 before you enter, they check your ID. If you're on the list, you get in. If not, you're redirected to the entrance desk. The room never even opens until the bouncer approves.

`CanDeactivate` is the coat check attendant who stops you on the way out \u2014 "Wait, did you leave anything behind? You have unsaved changes on that form \u2014 are you sure you want to leave?" You can still leave, but only after confirming.

`Resolve` is the venue staff who sets up the table before you're seated \u2014 by the time you walk in, the food is already on the table. The component receives fully-loaded data the moment it mounts. No loading spinner, no empty state.

The bouncer and the exit attendant control whether navigation happens. The setup crew controls what data is ready when it does.

Technical Explanation

Route guards are functions that the Router calls during the navigation lifecycle before committing to a route change. They return:

- `true` \u2014 allow navigation
- `false` \u2014 block navigation (stays on current URL)
- `UrlTree` \u2014 block and redirect to a different route
- `Observable<boolean | UrlTree>` or `Promise<boolean | UrlTree>` \u2014 async versions of the above

Navigation lifecycle order:

```
URL change triggered
  \u2192 CanDeactivate  (can we leave the current route?)
  \u2192 CanActivate    (can we enter the new route?)
```

```
\u2192 Resolve        (fetch data before component mounts)
\u2192 Component renders with data already available
```

Guards are registered directly in the route configuration \u2014 they run automatically on every matching navigation without any component knowing they exist.

Guard Types \u2014 Quick Reference

Guard	Runs When	Returns	Primary Use Case	
`canActivate`	Before entering a route	`boolean \	UrlTree`	Auth walls, role checks
`canActivateChild`	Before entering any child route	`boolean \	UrlTree`	Protect entire feature sections
`canDeactivate`	Before leaving a route	`boolean`	Unsaved changes warning	
`canMatch`	Before matching a route at all	`boolean`	Show different components per role	
`resolve`	After guards pass, before component mounts	any (the resolved data)	Pre-fetch required data	

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular guards are plain functions \u2014 not classes. The class-based `implements CanActivate` pattern is deprecated since Angular 15. Use `CanActivateFn`, `CanDeactivateFn`, and `ResolveFn` type aliases with `inject()` inside the function body.

```
// \u2705 Modern \u2014 functional guard
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn$.pipe(
    map(loggedIn => loggedIn || router.createUrlTree(['/login']))
  );
};

// \u274c Legacy \u2014 class-based guard (deprecated)
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(): Observable<boolean | UrlTree> { ... }
}
```

Migration table:

Legacy Pattern	Modern Replacement
`implements CanActivate` class	`CanActivateFn` function

Legacy Pattern	Modern Replacement
`implements CanDeactivate<T>` class	`CanDeactivateFn<T>` function
`implements Resolve<T>` class	`ResolveFn<T>` function
Constructor injection inside guard	`inject()` inside the guard function
`router.navigate()` to redirect	`router.createUrlTree()` returning a `UrlTree`

Section 3 \u2014 Code Example

Plain TS \u2014 guard as a decision function (concept in isolation)

```
// A guard is just a function that returns a decision
// The Router calls it and waits for the answer before proceeding
function canUserAccess(user: { role: string }): boolean {
  if (user.role === 'admin') {
    return true;            // allow navigation
  }
  return false;             // block \u2014 in real Angular, return a UrlTree to redirect
}
```

`guards/auth.guard.ts` \u2014 CanActivate with Observable auth state

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
import { map } from 'rxjs/operators';

export const authGuard: CanActivateFn = (route, state) => {
  const auth   = inject(AuthService);
  const router = inject(Router);

  return auth.isLoggedIn$.pipe(
    map(isLoggedIn => {
      if (isLoggedIn) return true;

      // Return a UrlTree \u2014 blocks navigation AND redirects atomically
      // This is NOT the same as calling router.navigate() \u2014 see Mistake 1
      return router.createUrlTree(['/login'], {
        queryParams: { returnUrl: state.url } // preserve intended destination
      });
    })
  );
};
```

`app.routes.ts` \u2014 attaching guards to routes

```
import { authGuard } from './guards/auth.guard';
import { roleGuard } from './guards/role.guard';

export const APP_ROUTES: Routes = [
  { path: 'login', component: LoginComponent },

  {
```

```
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [authGuard],               // single guard
  },
  {
    path: 'admin',
    component: AdminComponent,
    canActivate: [authGuard, roleGuard],    // both must return true \u2014 run sequentially
    data: { requiredRole: 'admin' },        // read by roleGuard via route.data
  },
  {
    path: 'settings',
    canActivateChild: [authGuard],          // protects ALL children without repeating guard
    children: [
      { path: 'profile', component: ProfileSettingsComponent },
      { path: 'billing', component: BillingSettingsComponent },
    ]
  }
];
```

`guards/unsaved-changes.guard.ts` \u2014 CanDeactivate

```
import { CanDeactivateFn } from '@angular/router';

// Interface contract: the guard asks the component about its own state
export interface HasUnsavedChanges {
  hasUnsavedChanges(): boolean;
}

export const unsavedChangesGuard: CanDeactivateFn<HasUnsavedChanges> =
  (component) => {
    if (component.hasUnsavedChanges()) {
      // In production: replace confirm() with a modal dialog returning Observable<boolean>
      return confirm('You have unsaved changes. Leave anyway?');
    }
    return true; // no unsaved changes \u2014 allow navigation freely
  };
```

`edit-profile.component.ts` \u2014 implementing the interface

```
@Component({ selector: 'app-edit-profile', standalone: true, ... })
export class EditProfileComponent implements HasUnsavedChanges {
  private fb   = inject(FormBuilder);
  private router = inject(Router);

  form = this.fb.group({ name: [''], email: [''] });

  isSaving = false; // flag to allow programmatic navigation past the guard

  // Guard calls this method before allowing navigation away
  hasUnsavedChanges(): boolean {
    if (this.isSaving) return false; // intentional programmatic navigation \u2014 skip check
    return this.form.dirty;
  }

  onSave(): void {
    this.isSaving = true;            // signal guard to allow the upcoming navigate()
    this.profileService.save(this.form.value).subscribe(() => {
```

```
      this.router.navigate(['/profile']);
    });
  }
}
```

`resolvers/user.resolver.ts` \u2014 ResolveFn with error handling

```
import { inject } from '@angular/core';
import { ResolveFn, Router } from '@angular/router';
import { UserService } from '../services/user.service';
import { User } from '../models/user.model';
import { catchError, EMPTY } from 'rxjs';

export const userResolver: ResolveFn<User> = (route) => {
  const userService = inject(UserService);
  const router      = inject(Router);
  const id          = route.paramMap.get('id')!;

  return userService.getUser(id).pipe(
    catchError(() => {
      // Always pair EMPTY with a navigate() \u2014 without it the user is left stranded
      router.navigate(['/not-found']);
      return EMPTY; // EMPTY completes without emitting \u2014 navigation is cancelled cleanly
    })
  );
};
```

`app.routes.ts` \u2014 attaching the resolver

```
{
  path: 'users/:id',
  component: UserDetailComponent,
  resolve: {
    user: userResolver,  // key 'user' is how the component accesses resolved data
  },
}
```

`user-detail.component.ts` \u2014 consuming resolved data

```
@Component({ ... })
export class UserDetailComponent {
  private route = inject(ActivatedRoute);

  // Synchronous access \u2014 resolver fetched this before component mounted
  // No loading state needed
  user = this.route.snapshot.data['user'] as User;

  // For reactive updates when navigating between records (component reused):
  user$ = this.route.data.pipe(
    map(data => data['user'] as User),
    takeUntilDestroyed()
  );
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 auth check inside the component

```
@Component({ ... })
export class DashboardComponent implements OnInit {
  private auth   = inject(AuthService);
  private router = inject(Router);

  ngOnInit(): void {
    // Auth check INSIDE the component \u2014 component mounts first, then checks
    if (!this.auth.isLoggedIn()) {
      this.router.navigate(['/login']);
    }
    // Problem: template renders before the check completes
    // Sensitive UI elements flash on screen for logged-out users
  }
}
```

What breaks: The component mounts first, the template renders (including any sensitive data), and then the redirect fires. Users with browser dev tools can see the flash of protected content. Auth logic scattered across every protected component also creates maintenance debt \u2014 change the auth rule and you must update every component individually.

\u2705 After \u2014 auth check in a guard

```
// Guard runs BEFORE component is created
// Component constructor, ngOnInit, and template never execute unless guard returns true
export const authGuard: CanActivateFn = () => {
  const auth   = inject(AuthService);
  const router = inject(Router);

  return auth.isLoggedIn$.pipe(
    map(isLoggedIn => isLoggedIn || router.createUrlTree(['/login']))
  );
};
```

Why this matters: Guards run in the Router's navigation pipeline before any component lifecycle begins. The component's constructor, `ngOnInit`, and template never execute unless the guard returns `true`. This is the correct architectural boundary \u2014 components handle UI, guards handle access policy.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Calling `router.navigate()` inside a guard instead of returning a `UrlTree`

What it is: Calling `this.router.navigate(['/login'])` inside a guard and returning `false`.

Why it happens: It feels like the natural way to redirect \u2014 the same pattern used in components.

How to avoid: Always return `router.createUrlTree(['/login'])`. This is a `UrlTree`, not a navigation call \u2014 the Router handles the redirect as part of the same navigation event.

Angular consequence: Calling `navigate()` inside a guard triggers a second navigation event while the first is still in progress. This causes race conditions, duplicate navigation entries in browser history, and "Navigation ID X is not equal to current ID" console warnings.

Mistake 2 \u2014 Returning `EMPTY` from a resolver without redirecting

What it is: `return EMPTY` in a resolver when a resource isn't found, without navigating elsewhere first.

Why it happens: `EMPTY` stops the stream cleanly \u2014 it feels like the right "nothing to return" value.

How to avoid: Always pair `return EMPTY` with a `router.navigate(['/not-found'])` call before it. The navigate call goes first, then `EMPTY` cancels the current navigation.

Angular consequence: `EMPTY` completes the resolver Observable without emitting a value. The Router treats this as "resolver cancelled navigation" \u2014 the URL changes but the component never mounts, leaving the user on a blank page with no feedback.

Mistake 3 \u2014 Treating `canActivate` returning `false` as a redirect

What it is: Returning `false` from a guard and expecting the user to land somewhere useful.

Why it happens: "False means blocked, so the Router will handle it" \u2014 but the Router just cancels and stays put.

How to avoid: Always return a `UrlTree` when you want to redirect. Return `false` only when you want navigation blocked with no redirect \u2014 a rare edge case.

Angular consequence: The URL in the address bar reverts to the previous URL silently. Users are confused \u2014 they clicked a link and nothing happened. No error, no explanation.

Mistake 4 \u2014 Using class-based guards in new code

What it is: Writing `@Injectable() export class AuthGuard implements CanActivate` in new Angular code.

Why it happens: Most pre-2023 tutorials, Stack Overflow answers, and legacy codebases use the class pattern \u2014 it's still everywhere.

How to avoid: Always write guards as `CanActivateFn` functions with `inject()`. Class-based guards are deprecated since Angular 15 and will eventually be removed.

Angular consequence: Works today but creates tech debt. Class-based guards can't use `inject()` contextually, are harder to tree-shake, and will need to be migrated when Angular drops support.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Return `router.createUrlTree(['/login'])` from guards to redirect	Call `router.navigate()` inside a guard \u2014 triggers a race condition
Write guards as plain `CanActivateFn` functions using `inject()`	Write guards as classes with `implements CanActivate` \u2014 deprecated
Redirect to a `returnUrl` query param so users land back after login	Always redirect to `/home` after login \u2014 users lose their intended destination
Use `resolve` to pre-fetch data the component always needs on mount	Use `resolve` for optional data \u2014 it delays navigation even when data isn't critical
Implement `HasUnsavedChanges` interface on components guarded by `canDeactivate`	Read component state directly inside a guard \u2014 breaks separation of concerns
Always `catchError` in resolvers and redirect to a 404 page	Let resolver errors propagate \u2014 the Router silently cancels navigation
Use `canActivateChild` on a parent route to protect all children at once	Repeat the same guard on every individual child route
Use `route.data` to pass config (like `requiredRole`) into guards	Hardcode role strings inside the guard function \u2014 not reusable

---

Section 7 \u2014 Interview Prep

Question 1

"What's the difference between returning `false` and returning a `UrlTree` from a `CanActivate` guard?"

Strong answer covers:

- `false` blocks navigation and leaves the user on the current URL with no indication of what happened
- `UrlTree` blocks navigation and triggers a redirect to the specified route as part of the same navigation event
- `router.createUrlTree(['/login'])` is the correct way to create a `UrlTree`
- Returning `false` is only appropriate when blocking is intentional and the user is already in the right place
- Calling `router.navigate()` inside a guard instead of returning `UrlTree` causes a race condition \u2014 two navigations running simultaneously, producing "Navigation ID X is not equal to current ID" warnings

Weak answer misses: The race condition caused by calling `navigate()` inside a guard instead of returning a `UrlTree`.

Level: Mid

Question 2

"Why would you use a `Resolve` guard instead of fetching data inside `ngOnInit`? What are the trade-offs?"

Strong answer covers:

- Resolver runs before the component mounts \u2014 data is available synchronously in `snapshot.data`
- Eliminates loading states and empty template flashes for fast API calls
- Resolver errors can redirect before the component ever exists \u2014 cleaner handling for "resource not found"
- Trade-off: user sees no navigation progress for slow resolvers \u2014 previous page appears frozen
- Trade-off: removes the component's ability to show its own loading or error UI
- Best for fast calls, required data, and routes where a blank intermediate state would hurt UX
- Combine both patterns: Resolve for critical data, in-component loading for supplementary data

Weak answer misses: The UX trade-off of slow resolvers \u2014 treating Resolve as universally superior to in-component loading.

Level: Mid\u2013Senior

Question 3

"How would you protect an entire feature section (multiple child routes) with a single auth guard without repeating it on every route?"

Strong answer covers:

- Use a parent route with `canActivateChild` \u2014 the guard runs before each child navigation
- Child routes inherit the parent's guard \u2014 no repetition needed
- Key distinction: `canActivate` on the parent runs once when entering the parent; `canActivateChild` runs on every child navigation even within an already-entered parent

```
{
  path: 'admin',
  canActivateChild: [authGuard], // runs for EVERY child navigation
  children: [
    { path: 'users',    component: AdminUsersComponent },
    { path: 'settings', component: AdminSettingsComponent },
  ]
}
```

Weak answer misses: The distinction between `canActivate` and `canActivateChild` \u2014 applying the guard to the parent with `canActivate` only checks once when the parent is first entered, not on every subsequent child navigation.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Route guards are plain functions that run inside the Router's navigation pipeline before any component mounts. They control whether navigation proceeds, redirects, or pre-loads data. `CanActivate` guards entry, `CanDeactivate` guards exit, and `Resolve` prepares data before the component is created.

Syntax at a glance:

Scenario	Guard / Pattern	Return Type	
Require login	`canActivate: [authGuard]`	`Observable<true \	UrlTree>`
Role-based access	`canActivate: [roleGuard]`, `data: { role }`	`Observable<true \	UrlTree>`
Protect all children	`canActivateChild: [authGuard]`	`Observable<true \	UrlTree>`
Unsaved form warning	`canDeactivate: [unsavedGuard]`	`boolean \	Observable<boolean>`
Pre-fetch required data	`resolve: { key: myResolver }`	`Observable<T>`	
Redirect on 404 data	Resolver `catchError` \u2192 `router.navigate()` + `return EMPTY`	`EMPTY`	

When to use / when not to use:

Use `canActivate` for every auth wall and role check \u2014 it is the most common guard. Use `canActivateChild` on a parent route when the same check applies to all children \u2014 avoids repetition. Use `canDeactivate` for unsaved-changes confirmation \u2014 do not use it to silently block navigation with no user feedback. Use `resolve` for fast, required data where a loading flash would hurt UX; do not use it for slow calls or optional data.

The one rule that matters most:

Never call `router.navigate()` inside a guard \u2014 always return `router.createUrlTree(['/destination'])` to redirect. Calling `navigate()` inside a guard triggers

a second simultaneous navigation and causes race conditions.

LAZY LOADING

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The configuration is concise, but understanding why it works requires grasping how JavaScript module bundling, the browser's network loading model, and Angular's dependency injection system all interact at the route boundary.

Angular Relevance: Critical \u2014 Every production Angular app with more than a handful of routes should use lazy loading. It is the single highest-impact performance optimisation available at the architectural level, and Angular's Router makes it nearly free to implement.

Section 1 \u2014 Explanation

The Library Branch Analogy

Imagine a large public library system with a central branch and several specialist branches \u2014 law, medicine, science, children's.

Without lazy loading: Every time anyone walks into the central library, staff wheel out every book from every branch and stack them in the lobby \u2014 law books, medical textbooks, children's picture books, all of it \u2014 just in case someone might ask for them. The lobby is packed, nobody can move, and the first patron to arrive waits three minutes before they can even ask a question.

With lazy loading: The central library keeps only its own books in the lobby. The specialist branches stay at their own locations. When a patron specifically asks for a law book, a courier is dispatched to the law branch, retrieves just what's needed, and brings it back. First patrons arrive instantly. People who never need the law section never pay the cost of fetching those books.

In Angular terms: the central library is your initial JavaScript bundle \u2014 everything loaded on first visit. Each specialist branch is a feature area (admin panel, settings, reports). The courier dispatch is `loadComponent` / `loadChildren` in your route config. The books arriving are JavaScript chunk files downloaded on demand from your server.

Technical Explanation

When you build an Angular app, the compiler bundles your TypeScript into JavaScript files. Without lazy loading, everything goes into one `main.js` \u2014 every component, every service, every dependency. The browser must download and parse all of it before the app becomes interactive.

Lazy loading tells the bundler (Webpack or esbuild): "This component is only needed when the user navigates to this route \u2014 put it in a separate chunk file." The browser downloads that chunk only when the route is first visited.

Two modern approaches in Angular v17+:

- `loadComponent` \u2192 lazily loads a single standalone component
- `loadChildren` \u2192 lazily loads an entire routes file (feature routes)

Both use the dynamic `import()` syntax \u2014 a standard JavaScript feature that tells the bundler to split at that boundary.

What happens at runtime:

```
User visits /            \u2192 browser downloads main.js only
User navigates to /admin \u2192 Router sees loadComponent, triggers dynamic import()
                         \u2192 browser downloads admin-dashboard-HASH.js chunk
                         \u2192 Router instantiates AdminDashboardComponent in <router-outlet>
Subsequent /admin visits \u2192 chunk already cached \u2014 instant
```

Lazy Loading Approaches \u2014 Quick Reference

Approach	Syntax	Best For	Chunk Contains
`loadComponent`	`loadComponent: () => import('./x').then(m => m.XComponent)`	Single standalone component	That component + its unique deps
`loadChildren`	`loadChildren: () => import('./x.routes').then(m => m.X_ROUTES)`	Entire feature section	All components in that routes file
Eager (default)	`component: MyComponent`	Shell, login, home	Added to main.js
Preloading	`withPreloading(PreloadAllModules)`	Routes likely to be visited	Silently prefetched after initial load

---

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular lazy loading uses `loadComponent` and `loadChildren` with dynamic `import()` \u2014 no NgModules, no `loadChildren` string syntax. The route's `providers` array replaces NgModule-level service scoping.

Migration table:

Legacy Pattern	Modern Replacement
`loadChildren: './admin/admin.module#AdminModule'` (string)	`loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES)`
`loadChildren: () => import('./x').then(m => m.XModule)` (NgModule)	`loadChildren: () => import('./x.routes').then(m => m.X_ROUTES)` (Routes array)
Services scoped via NgModule	`providers: [MyService]` in the route config

Legacy Pattern	Modern Replacement
`RouterModule.forChild(routes)` inside NgModule	Direct `Routes` array exported from the feature routes file

---

Section 3 \u2014 Code Example

Plain JS \u2014 dynamic import in isolation (the underlying mechanism)

```
// Static import \u2014 loaded immediately, part of the main bundle
import { AdminDashboard } from './admin-dashboard';

// Dynamic import \u2014 returns a Promise, loaded on demand
// This is standard JavaScript (ES2020) \u2014 Angular's lazy loading is built on this
async function loadAdminWhenNeeded() {
  const module = await import('./admin-dashboard'); // only called when this runs
  return module.AdminDashboard;
}
// The bundler sees the dynamic import() and creates a separate chunk file
// That chunk is only downloaded when loadAdminWhenNeeded() is called
```

`app.routes.ts` \u2014 `loadComponent` for single components

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component'; // eager \u2014 in main bundle
import { authGuard } from './guards/auth.guard';

export const APP_ROUTES: Routes = [
  // Eager \u2014 HomeComponent included in main.js (fast first load)
  { path: 'home', component: HomeComponent },

  // Lazy \u2014 AdminDashboardComponent in its own chunk
  // The () => import(...) is NOT called until the user navigates here
  {
    path: 'admin',
    loadComponent: () =>
      import('./admin/admin-dashboard.component')
        .then(m => m.AdminDashboardComponent),
    canActivate: [authGuard], // guard runs BEFORE the chunk is downloaded
  },

  // Lazy with default export \u2014 simpler .then() not needed
  {
    path: 'settings',
    loadComponent: () => import('./settings/settings.component'),
    // Works when the component is the default export of the file
  },

  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: '**', loadComponent: () => import('./not-found.component') },
];
```

`app.routes.ts` \u2014 `loadChildren` for entire feature sections

```
export const APP_ROUTES: Routes = [
```

```
  { path: 'home', component: HomeComponent },

  // Entire /users feature section is lazy
  // users.routes.ts AND ALL its components go in a separate chunk
  {
    path: 'users',
    loadChildren: () =>
      import('./users/users.routes')
        .then(m => m.USERS_ROUTES),
    canActivate: [authGuard],
  },

  { path: '', redirectTo: 'home', pathMatch: 'full' },
];
```

`users/users.routes.ts` \u2014 feature routes file

```
import { Routes } from '@angular/router';

// This entire file (and everything it imports) becomes a separate JS chunk
export const USERS_ROUTES: Routes = [
  {
    path: '',            // matches /users exactly
    loadComponent: () =>
      import('./user-list.component').then(m => m.UserListComponent),
  },
  {
    path: ':id',         // matches /users/42
    loadComponent: () =>
      import('./user-detail.component').then(m => m.UserDetailComponent),
  },
  {
    path: ':id/edit',    // matches /users/42/edit
    loadComponent: () =>
      import('./user-edit.component').then(m => m.UserEditComponent),
    canDeactivate: [unsavedChangesGuard],
  },
];
```

`app.config.ts` \u2014 registering routes with preloading

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { APP_ROUTES } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(
      APP_ROUTES,
      // Silently prefetch lazy chunks after initial load completes
      // User gets instant navigation to lazy routes after ~2s
      withPreloading(PreloadAllModules),
    ),
  ],
};
```

Route-level service scoping with `providers`

```
// users/users.routes.ts
export const USERS_ROUTES: Routes = [
  {
    path: '',
    // providers array creates a scoped DI context for this route subtree
    // UserStateService here is a DIFFERENT instance from any root-level service
    providers: [UserStateService],
    children: [
      {
        path: '',
        loadComponent: () =>
          import('./user-list.component').then(m => m.UserListComponent),
      },
      {
        path: ':id',
        loadComponent: () =>
          import('./user-detail.component').then(m => m.UserDetailComponent),
      },
    ],
  },
];
```

Auth-gated lazy loading \u2014 full production pattern

```
// app.routes.ts \u2014 guard runs BEFORE the chunk downloads
export const APP_ROUTES: Routes = [
  // Eagerly loaded \u2014 needed by everyone immediately
  { path: '',      component: LandingPageComponent },
  { path: 'login', component: LoginComponent },

  // Admin section \u2014 gated by auth, entire bundle deferred
  {
    path: 'app',
    canActivate: [authGuard],       // 1. Check auth FIRST
    loadChildren: () =>             //    If blocked, chunk is NEVER downloaded
      import('./portal/portal.routes')
        .then(m => m.PORTAL_ROUTES), // 2. THEN download the portal chunk
  },

  { path: '**', loadComponent: () => import('./not-found.component') },
];

// portal/portal.routes.ts \u2014 nested lazy splits within the portal chunk
export const PORTAL_ROUTES: Routes = [
  {
    path: '',
    component: PortalShellComponent, // shell is eager within the portal chunk
    children: [
      {
        path: 'dashboard',
        loadComponent: () =>
          import('./dashboard/dashboard.component').then(m => m.DashboardComponent),
      },
      {
        path: 'users',
        loadChildren: () =>
          import('./users/users.routes').then(m => m.USERS_ROUTES),
      },
      {
```

```
        path: 'reports',
        loadChildren: () =>
          import('./reports/reports.routes').then(m => m.REPORTS_ROUTES),
      },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
    ],
  },
];
// Result: unauthenticated users never download portal JS.
// Authenticated users download portal shell on first login,
// then individual section chunks on demand.
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 everything eager, one giant bundle

```
// app.routes.ts \u2014 all components imported statically at the top
import { HomeComponent }           from './home/home.component';
import { AdminDashboardComponent } from './admin/admin-dashboard.component';
import { AdminUsersComponent }     from './admin/admin-users.component';
import { AdminReportsComponent }   from './admin/admin-reports.component';
import { SettingsComponent }       from './settings/settings.component';
import { UserListComponent }       from './users/user-list.component';
import { UserDetailComponent }     from './users/user-detail.component';

export const APP_ROUTES: Routes = [
  { path: 'home',         component: HomeComponent },
  { path: 'admin',        component: AdminDashboardComponent },
  { path: 'admin/users',  component: AdminUsersComponent },
  { path: 'admin/reports',component: AdminReportsComponent },
  { path: 'settings',     component: SettingsComponent },
  { path: 'users',        component: UserListComponent },
  { path: 'users/:id',    component: UserDetailComponent },
];
```

What breaks: Every component is statically imported at the top of `app.routes.ts`. The bundler sees these imports and includes all component code in `main.js`. A user visiting only the home page downloads the entire admin panel \u2014 all its templates, logic, and dependencies \u2014 immediately. On a slow 3G connection, this adds seconds to first load.

\u2705 After \u2014 feature sections lazy-loaded

```
// app.routes.ts \u2014 NO static imports of feature components
// Only shell/home components imported eagerly
import { HomeComponent } from './home/home.component';

export const APP_ROUTES: Routes = [
  { path: 'home', component: HomeComponent }, // small, needed immediately

  {
    path: 'admin',
    canActivate: [authGuard],
    loadChildren: () =>
      import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
    // ~0kb in main.js \u2014 downloaded only if user navigates to /admin
```

```
  },

  {
    path: 'users',
    loadChildren: () =>
      import('./users/users.routes').then(m => m.USERS_ROUTES),
  },

  {
    path: 'settings',
    loadComponent: () => import('./settings/settings.component'),
  },
];
```

Why this matters: The Angular compiler uses static import analysis to determine what goes in the bundle. Replacing `import { X } from './x'` at the top of the file with `() => import('./x')` inside a route config is the signal that tells the bundler to split at that boundary. The change is minimal \u2014 the impact on initial load time in large apps is significant. Angular's build output will show separate chunk files for each lazy route.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Statically importing the component AND using `loadComponent`

What it is: `import { AdminComponent } from './admin.component'` at the top of the file, then `loadComponent: () => import('./admin.component').then(...)` in the route.

Why it happens: IDE auto-import adds the static import when you type the component name. The developer doesn't notice and also adds the lazy route.

How to avoid: If you use `loadComponent` or `loadChildren`, there must be no static import of those components at the top of `app.routes.ts`. Remove any auto-added imports immediately.

Angular consequence: The static import causes the bundler to include the component in `main.js` anyway \u2014 the `loadComponent` call still works but the lazy split never happens. Zero performance benefit, no error, no warning from Angular.

Mistake 2 \u2014 Providing a service in both root and a lazy route's `providers` array

What it is: `@Injectable({ providedIn: 'root' })` on a service that is also listed in a lazy route's `providers: [MyService]`.

Why it happens: Developers add `providers` to the route for scoping without removing `providedIn: 'root'` from the service decorator.

How to avoid: Choose one: either `providedIn: 'root'` (one singleton for the whole app) or `providers: [MyService]` on the route (scoped instance). Never both.

Angular consequence: Two instances of the service exist simultaneously. Components in the lazy route get the scoped instance; components outside it get the root instance. State updates in

one are invisible to the other \u2014 extremely hard to debug.

Mistake 3 \u2014 Using `loadChildren` pointing to a component instead of a routes file

What it is: `loadChildren: () => import('./user-list.component').then(m => m.UserListComponent)` \u2014 passing a component class where a `Routes` array is expected.

Why it happens: Both `loadComponent` and `loadChildren` use `() => import(...)` syntax \u2014 they look identical. Developers confuse which resolves to what.

How to avoid: `loadComponent` resolves to a component class. `loadChildren` resolves to a `Routes` array. Check what the imported file actually exports.

Angular consequence: The Router expects a `Routes` array from `loadChildren`. Receiving a component class causes a runtime error or silent blank render: `Error: Invalid configuration of route 'users'.`

Mistake 4 \u2014 Assuming lazy chunks are re-downloaded on every navigation

What it is: Adding artificial caching logic or worrying about performance on repeated visits to a lazy route.

Why it happens: "Lazy" sounds like "slow every time" \u2014 developers assume the chunk is re-fetched on each visit.

How to avoid: Dynamic `import()` is cached by the browser's module registry after the first download. Second navigation to a lazy route is as fast as an eager route.

Angular consequence: Unnecessary complexity \u2014 cache-busting logic, service workers configured to avoid lazy chunks, manual preloading of already-cached chunks. Wasted dev time and potential bugs.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Use `loadChildren` for entire feature sections with multiple routes	Import feature components statically in `app.routes.ts` \u2014 it defeats the lazy split
Use `loadComponent` for single large components that are rarely visited	Use `loadComponent` for tiny components \u2014 HTTP round-trip overhead outweighs savings
Keep eagerly loaded components minimal: shell, nav, login, home	Lazy-load login or root `AppComponent` \u2014 they're needed immediately on every visit
Add `canActivate: [authGuard]` before `loadChildren` \u2014 guard blocks chunk download	Download the admin bundle before checking if the user is authorised

\u2705 DO	\u274c DON'T
Use `withPreloading(PreloadAllModules)` to background-fetch chunks after initial load	Assume all users have fast connections \u2014 preloading matters on slow networks
Scope feature-specific services in the route's `providers` array	Use `providedIn: 'root'` for services that should only live inside one lazy feature
Verify lazy splitting by checking `ng build` chunk output	Trust that lazy loading is working without confirming in build output

---

Section 7 \u2014 Interview Prep

Question 1

"What is lazy loading in Angular and how does it affect the build output?"

Strong answer covers:

- Lazy loading defers downloading JavaScript for a route until the user navigates to it
- Implemented via `loadComponent` or `loadChildren` with dynamic `import()` syntax
- The Angular bundler (webpack/esbuild) creates separate chunk files for each lazy boundary
- `main.js` only contains eagerly-loaded code \u2014 everything behind `loadComponent`/`loadChildren` is in named chunks
- First visit to a lazy route downloads its chunk; subsequent visits use the browser cache
- Practical impact: smaller initial bundle \u2192 faster Time to Interactive (TTI)
- A static import of the component at the top of the routes file silently defeats the split \u2014 no error is thrown

Weak answer misses: How the bundler knows to split \u2014 the static vs dynamic `import()` distinction \u2014 and whether they understand it's about build output, not just runtime behaviour.

Level: Junior\u2013Mid

Question 2

"How do route guards interact with lazy-loaded routes? Does the guard prevent the chunk from being downloaded?"

Strong answer covers:

- Guards in `canActivate` run before `loadChildren`/`loadComponent` is called
- A guard returning `false` or a `UrlTree` means the dynamic `import()` is never triggered \u2014 the chunk is not downloaded
- This is a security benefit: unauthorized users never download admin or protected code

- Angular handles the ordering automatically \u2014 the navigation pipeline runs guards first, then fetches the chunk
- Contrast with putting auth logic inside the component: the component bundle would already be downloaded before the check runs

Weak answer misses: Explicitly confirming the chunk is not downloaded on guard failure \u2014 many candidates say "the guard blocks navigation" without addressing whether the network request fires.

Level: Mid\u2013Senior

Question 3

"What is a preloading strategy and when would you use a custom one over `PreloadAllModules`?"

Strong answer covers:

- Preloading silently downloads lazy chunks after the initial app load, during browser idle time
- `PreloadAllModules` downloads all lazy chunks \u2014 good for small apps where bandwidth is not a concern
- Custom strategies (implementing `PreloadingStrategy`) allow selective preloading based on route `data` flags, user role, or network conditions (`navigator.connection`)
- `withPreloading()` in `provideRouter()` registers the strategy
- Trade-off: `PreloadAllModules` on a large app with many chunks wastes bandwidth on mobile, where users may only visit one or two routes

Weak answer misses: The custom strategy use case \u2014 just saying "PreloadAllModules downloads everything" without discussing when selective preloading is the better choice.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Lazy loading splits your app into separate JavaScript chunk files that are downloaded only when the user navigates to that route. `loadComponent` lazily loads one component; `loadChildren` lazily loads an entire routes file. The mechanism is standard JavaScript dynamic `import()` \u2014 Angular's Router exploits module boundaries to create bundle split points.

Syntax at a glance:

Task	API
Lazy-load one component	`loadComponent: () => import('./x').then(m => m.XComponent)`

Task	API
Lazy-load a feature routes file	`loadChildren: () => import('./x.routes').then(m => m.X_ROUTES)`
Default export shorthand	`loadComponent: () => import('./x.component')`
Enable preloading	`provideRouter(routes, withPreloading(PreloadAllModules))`
Scope a service to a route	`providers: [MyService]` in the route config
Verify lazy splitting	`ng build` \u2014 check for separate chunk files in output

`loadComponent` **vs** `loadChildren`**:**

Eager (`component`)	`loadComponent`	`loadChildren`	
In `main.js`	\u2705 Yes	\u274c No	\u274c No
Separate chunk	No	Yes (one component)	Yes (whole feature)
Guard interaction	N/A	Guard first, then render	Guard first, then download
Best for	Shell, login, home	Single large page	Feature sections
Child routes	Yes	No	Yes
DI scoping	Root only	Add `providers` to route	Add `providers` to route

When to use / when not to use:

Use `loadChildren` for large feature sections with multiple routes (admin, reports, billing). Use `loadComponent` for single large components that are rarely visited (help pages, legal, onboarding). Do not lazy-load small components or components needed on first load \u2014 the HTTP round-trip for a 3KB chunk costs more than including it in `main.js`.

The one rule that matters most:

A static import of a lazy component in your routes file silently defeats the entire lazy split \u2014 always remove auto-added imports from `app.routes.ts` and verify chunks appear in the build output.

Now for the next topic.

Your learning order has Routing Fundamentals, Route Guards, and Lazy Loading done. Next is **Nested Routes & Layout Components** (Topic 36 in your plan).

NESTED ROUTES & LAYOUT COMPONENTS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The `children` array syntax is simple, but understanding how multiple `<router-outlet>` slots interact, how parent params flow to child components, and how layout components create persistent shells requires a solid mental model of the Router tree.

Angular Relevance: Critical \u2014 Every real-world Angular app uses nested routes. Dashboard layouts, admin panels, tabbed feature sections, and persistent sidebars are all built on this pattern. A flat route structure doesn't scale beyond a handful of pages.

Section 1 \u2014 Explanation

The Office Building Analogy

Think of a flat route structure as a single-floor office where every room opens directly off the street. Every time you visit any room, you walk in from outside, the entire lobby resets, and nothing persists between visits.

Nested routes are a multi-storey office building. The ground floor is the shell \u2014 the reception desk, the elevator, the company logo on the wall. These never change regardless of which floor you're on. Each floor above is a feature area (HR, Finance, Engineering). Each floor has its own corridors (child routes) and rooms (leaf components). When you take the elevator to Finance, the ground floor doesn't disappear \u2014 it stays. Only the floor's interior changes.

In Angular: the ground floor is your layout component (sidebar + header + `<router-outlet>`). The floors are feature sections. The rooms are the actual page components that render inside the outlet. The elevator is the Router navigating between child routes without touching the shell.

Technical Explanation

A nested route is a route with a `children` array. Each child route's path is relative to its parent. The parent component must have a `<router-outlet>` in its template \u2014 that's where child components render.

```
URL: /admin/users/42

Route tree:
AppComponent          <-- root <router-outlet>
  AdminShellComponent <-- /admin layout, has its own <router-outlet>
    UserDetailComponent <-- /admin/users/42 renders here
```

The root `AppComponent` has a `<router-outlet>` where `AdminShellComponent` renders. `AdminShellComponent` has its own `<router-outlet>` where `UserDetailComponent` renders. Two outlets, two levels, one URL.

Key terms:

Term	What It Is
Layout component	A component with a `<router-outlet>` that persists across child navigations
Shell component	The top-level layout \u2014 usually header + sidebar + outlet
Child route	A route nested inside a parent's `children` array
Relative path	Child paths don't start with `/` \u2014 they're relative to the parent path
`router-outlet`	The slot where the matched child component renders
Empty path parent	A parent route with `path: ''` used purely for layout, adding nothing to the URL

Section 2 \u2014 Modern Angular Pattern (v17+)

Nested routes in modern Angular use standalone components throughout \u2014 no NgModules. Layout components are standalone with `RouterOutlet` imported explicitly. Guards on parent routes protect all children. `loadChildren` on the parent lazy-loads the entire feature including its layout.

```
// \u2705 Modern \u2014 standalone layout component
@Component({
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>...</nav>
    <router-outlet />   <!-- child components render here -->
  `
})
export class AdminShellComponent {}
```

Migration from legacy:

Legacy Pattern	Modern Replacement
NgModule with `RouterModule.forChild(routes)`	Standalone `Routes` array in feature routes file
Layout component declared in NgModule	Standalone layout component with `RouterOutlet` in `imports`
`ActivatedRoute` in constructor	`inject(ActivatedRoute)` in field initializer
`this.route.parent.params.subscribe(...)`	`inject(ActivatedRoute).parent!.paramMap`

Section 3 \u2014 Code Example

Plain TS \u2014 nested structure in isolation

```
// Route nesting is just an array inside an array
// Child paths are RELATIVE \u2014 they don't start with /
const routes = [
  {
    path: 'admin',              // matches /admin
    component: 'AdminShell',    // renders in root <router-outlet>
    children: [
      { path: '',               component: 'AdminDashboard' },  // /admin
      { path: 'users',          component: 'UserList' },        // /admin/users
      { path: 'users/:id',      component: 'UserDetail' },      // /admin/users/42
      { path: 'settings',       component: 'Settings' },        // /admin/settings
    ]
  }
];
// AdminShell renders once and persists across all /admin/* navigations
// Only the component inside AdminShell's <router-outlet> changes
```

Full working example \u2014 admin portal with layout

admin/admin.routes.ts

```
import { Routes } from '@angular/router';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    // AdminShellComponent is the layout \u2014 header + sidebar + outlet
    // It renders in the ROOT <router-outlet> of AppComponent
    loadComponent: () =>
      import('./admin-shell.component').then(m => m.AdminShellComponent),
    children: [
      // /admin \u2192 redirect to /admin/dashboard
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' },

      // /admin/dashboard
      {
        path: 'dashboard',
        loadComponent: () =>
          import('./dashboard/dashboard.component').then(m => m.DashboardComponent),
      },

      // /admin/users \u2192 UserListComponent
      // /admin/users/42 \u2192 UserDetailComponent
      {
        path: 'users',
        loadComponent: () =>
          import('./users/user-list.component').then(m => m.UserListComponent),
      },
      {
        path: 'users/:id',
        loadComponent: () =>
          import('./users/user-detail.component').then(m => m.UserDetailComponent),
      },

      // /admin/settings
      {
        path: 'settings',
        loadComponent: () =>
```

```
            import('./settings/settings.component').then(m => m.SettingsComponent),
        },
      ],
    },
  ];
```

`app.routes.ts` \u2014 root routes

```
  import { Routes } from '@angular/router';
  import { HomeComponent } from './home/home.component';
  import { authGuard } from './guards/auth.guard';

  export const APP_ROUTES: Routes = [
    { path: 'home', component: HomeComponent },

    // Guard on the parent protects ALL children automatically
    // The admin chunk is only downloaded after the guard passes
    {
      path: 'admin',
      canActivate: [authGuard],
      loadChildren: () =>
        import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
    },

    { path: '', redirectTo: 'home', pathMatch: 'full' },
    { path: '**', loadComponent: () => import('./not-found.component') },
  ];
```

`admin/admin-shell.component.ts` \u2014 the layout component

```
  import { Component, inject } from '@angular/core';
  import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';
  import { AuthService } from '../services/auth.service';

  @Component({
    selector: 'app-admin-shell',
    standalone: true,
    // RouterOutlet MUST be imported \u2014 it won't work without it
    imports: [RouterOutlet, RouterLink, RouterLinkActive],
    template: `
      <div class="admin-layout">
        <!-- Sidebar \u2014 persists across ALL /admin/* navigations -->
        <aside class="sidebar">
          <nav>
            <a routerLink="dashboard" routerLinkActive="active">Dashboard</a>
            <a routerLink="users"     routerLinkActive="active">Users</a>
            <a routerLink="settings"  routerLinkActive="active">Settings</a>
          </nav>
          <button (click)="logout()">Logout</button>
        </aside>

        <!-- Main content area \u2014 child component renders here -->
        <main class="content">
          <router-outlet />
        </main>
      </div>
    `,
  })
```

```
  export class AdminShellComponent {
    private auth = inject(AuthService);

    logout(): void {
      this.auth.logout();
    }
  }
  // Note: routerLink="dashboard" (no leading slash) = relative to current route /admin
  // routerLink="/admin/dashboard" (with slash) = absolute \u2014 both work, relative is preferred
```

`admin/users/user-detail.component.ts` \u2014 reading parent route param from a child

```
  import { Component, inject } from '@angular/core';
  import { ActivatedRoute } from '@angular/router';
  import { toSignal } from '@angular/core/rxjs-interop';
  import { switchMap, map } from 'rxjs/operators';

  @Component({
    selector: 'app-user-detail',
    standalone: true,
    template: `
      @if (user()) {
        <h2>{{ user()!.name }}</h2>
        <p>{{ user()!.email }}</p>
      }
    `,
  })
  export class UserDetailComponent {
    private route = inject(ActivatedRoute);
    private userService = inject(UserService);

    // :id is on THIS route \u2014 read directly from paramMap
    user = toSignal(
      this.route.paramMap.pipe(
        map(p => p.get('id')!),
        switchMap(id => this.userService.getUser(+id))
      )
    );
  }

  // If :id were on the PARENT route instead (e.g. /admin/:adminId/users),
  // you'd access it via:
  // this.route.parent!.paramMap.pipe(map(p => p.get('adminId')!))
```

Empty-path parent \u2014 layout without adding to the URL

```
  // Sometimes you want a layout component that adds NO path segment
  // This is the "pathless layout" pattern \u2014 common for auth vs public sections

  export const APP_ROUTES: Routes = [
    // Public layout \u2014 wraps marketing pages
    {
      path: '',                     // adds nothing to the URL
      loadComponent: () =>
        import('./layouts/public-layout.component').then(m => m.PublicLayoutComponent),
      children: [
        { path: '',        component: HomeComponent },
        { path: 'about',   component: AboutComponent },
```

```
      { path: 'pricing', component: PricingComponent },
    ],
  },

  // App layout \u2014 wraps authenticated pages
  {
    path: 'app',
    canActivate: [authGuard],
    loadComponent: () =>
      import('./layouts/app-layout.component').then(m => m.AppLayoutComponent),
    children: [
      { path: 'dashboard', loadComponent: () => import('./dashboard/dashboard.component')... },
      { path: 'profile',   loadComponent: () => import('./profile/profile.component')... },
    ],
  },
];
// PublicLayoutComponent renders for /, /about, /pricing \u2014 different header/footer
// AppLayoutComponent renders for /app/dashboard, /app/profile \u2014 sidebar + nav
// Two distinct layouts, zero duplication, one Router config
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 flat routes, layout duplicated in every component

```
// app.routes.ts \u2014 every route is flat
export const APP_ROUTES: Routes = [
  { path: 'admin/dashboard', component: AdminDashboardComponent },
  { path: 'admin/users',     component: AdminUsersComponent },
  { path: 'admin/settings',  component: AdminSettingsComponent },
];

// AdminDashboardComponent \u2014 template duplicates the shell
@Component({
  template: `
    <aside class="sidebar">          <!-- duplicated in every admin component -->
      <a routerLink="/admin/users">Users</a>
      <a routerLink="/admin/settings">Settings</a>
    </aside>
    <main>
      <h1>Dashboard</h1>
      <!-- actual content -->
    </main>
  `
})
export class AdminDashboardComponent {}

// AdminUsersComponent \u2014 same sidebar duplicated again
// AdminSettingsComponent \u2014 same sidebar duplicated again
```

What breaks: The sidebar HTML and logic is copied into every admin component. A change to the sidebar (add a nav item, change the logout logic) requires updating every admin component individually. The sidebar also re-renders on every navigation \u2014 there's no persistence, no transition, just a full redraw. Guards must be applied to every individual route separately.

\u2705 After \u2014 nested routes with a persistent layout

```
// app.routes.ts \u2014 one parent, all children inherit layout and guard
{
  path: 'admin',
  canActivate: [authGuard],                   // one guard \u2014 covers all children
  loadComponent: () =>
    import('./admin/admin-shell.component')
      .then(m => m.AdminShellComponent),      // layout lives here once
  children: [
    { path: 'dashboard', loadComponent: () => import('./dashboard.component')... },
    { path: 'users',     loadComponent: () => import('./users.component')... },
    { path: 'settings',  loadComponent: () => import('./settings.component')... },
  ],
}

// AdminShellComponent owns the sidebar \u2014 written once
// DashboardComponent, UsersComponent, SettingsComponent contain only their own content
```

Why this matters: The layout component renders once when entering `/admin` and persists for all child navigations. Child components contain only their own content \u2014 no sidebar duplication. Guards applied to the parent automatically cover every child. Adding a new admin page is one new entry in `children` \u2014 no boilerplate.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Forgetting `RouterOutlet` in the layout component's imports

What it is: Writing a layout component with `<router-outlet />` in the template but omitting `RouterOutlet` from the `imports` array.

Why it happens: `RouterOutlet` is a directive \u2014 it behaves like a component but must be explicitly imported in standalone components. In NgModule apps this was handled globally by `RouterModule`; in standalone apps it's per-component.

How to avoid: Any standalone component whose template contains `<router-outlet>`, `routerLink`, or `routerLinkActive` must import `RouterOutlet`, `RouterLink`, `RouterLinkActive` respectively.

Angular consequence: The `<router-outlet>` silently does nothing \u2014 no error is thrown, the layout renders, but child components never appear. The URL updates correctly but the outlet stays blank.

Mistake 2 \u2014 Using absolute paths in child route `routerLink` directives

What it is: Inside `AdminShellComponent`, writing `routerLink="/admin/users"` (absolute) instead of `routerLink="users"` (relative).

Why it happens: Absolute paths always work and feel more explicit \u2014 developers default to them without thinking about nesting.

How to avoid: Inside a layout component that lives at `/admin`, child links should use relative paths (`"users"`, `"dashboard"`) or `[routerLink]="['users']"`. This means the layout component doesn't need to know its own absolute path.

Angular consequence: Absolute paths work until the route is restructured \u2014 move the admin section from `/admin` to `/portal/admin` and every `routerLink="/admin/..."` across all child components breaks simultaneously. Relative paths survive restructuring with zero changes.

Mistake 3 \u2014 Trying to read a parent route's param using `snapshot` in a child

What it is: In a child component, calling `this.route.snapshot.paramMap.get('adminId')` to read a param that belongs to the parent route \u2014 and getting `null`.

Why it happens: `inject(ActivatedRoute)` in a child component gives you the child's `ActivatedRoute`, not the parent's. The child's paramMap only contains params defined on the child's path segment.

How to avoid: Traverse to the parent: `this.route.parent!.paramMap.pipe(map(p => p.get('adminId')))`. Or use `withComponentInputBinding()` if the param is on the same route segment as the component.

Angular consequence: `params.get('parentParam')` silently returns `null` \u2014 no error. The child component receives null data and either crashes or shows blank content, with no obvious indication that the param is on a different route level.

Mistake 4 \u2014 Adding a leading slash to child route `path` strings

What it is: Writing `{ path: '/users', component: ... }` inside a `children` array.

Why it happens: Top-level routes always start without a slash but it's not obvious this convention changes for children. The intent is clear \u2014 the mistake is in the syntax.

How to avoid: Child route paths are always relative \u2014 never start with `/`. The full URL is assembled by the Router concatenating parent + child paths.

Angular consequence: Angular throws `Error: 'users' in the Router configuration cannot start with a slash` at startup \u2014 this one at least is caught immediately, unlike most Router misconfigurations.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Import `RouterOutlet` explicitly in every standalone layout component	Assume `<router-outlet>` works without importing `RouterOutlet` \u2014 it silently does nothing
Use relative `routerLink` paths inside layout components (`"users"` not `"/admin/users"`)	Hardcode absolute paths in nested components \u2014 they break on route restructuring

\u2705 DO	\u274c DON'T
Use empty-path parent routes (`path: ''`) for layout-only components that add nothing to the URL	Add dummy path segments just to attach a layout \u2014 they pollute the URL
Apply `canActivate` or `canActivateChild` once on the parent to protect all children	Repeat the same guard on every individual child route
Use `this.route.parent!.paramMap` to read params from a parent route segment	Call `this.route.snapshot.paramMap.get()` in a child for a param defined on the parent
Use `redirectTo` with `pathMatch: 'full'` on the empty child path	Leave the empty child path unhandled \u2014 navigating to just `/admin` renders nothing
Combine `loadChildren` on the parent with `loadComponent` on each child for maximum splitting	Bundle all child components into one eager import inside the feature routes file

---

Section 7 \u2014 Interview Prep

Question 1

"What is a layout component in Angular routing and how does it differ from a regular routed component?"

Strong answer covers:

- A layout component is a component that contains a `<router-outlet>` \u2014 it acts as a persistent shell that child components render inside
- It renders when the parent route is activated and stays alive across all child navigations \u2014 only the content inside its outlet changes
- A regular routed component is a leaf \u2014 it renders inside another outlet and contains no outlet of its own
- Layout components own shared UI (sidebar, header, breadcrumbs) that would otherwise be duplicated across every child component
- In the route config, the layout component is attached to the parent route; child components are in the `children` array
- Guards on the parent protect all children \u2014 the layout component is the natural place to centralise access control

Weak answer misses: That the layout component persists across child navigations \u2014 many candidates describe it as "a component that wraps others" without explaining the persistence or the outlet mechanics.

Level: Mid

Question 2

"How do you access a route parameter defined on a parent route from inside a child component?"

Strong answer covers:

- `inject(ActivatedRoute)` in a child gives you the child's own `ActivatedRoute` \u2014 its `paramMap` only contains params from its own path segment
- To read a parent's param, traverse up: `this.route.parent!.paramMap.pipe(map(p => p.get('id')))`
- Can chain `.parent` multiple levels up for deeply nested routes
- `withComponentInputBinding()` only binds params from the component's own route segment \u2014 it doesn't help with parent params
- The common production pattern is to put shared params on a shared parent route and have child components traverse up explicitly

Weak answer misses: That `inject(ActivatedRoute)` is scoped to the component's own route level \u2014 candidates often expect it to see all params in the URL regardless of which segment they come from.

Level: Mid\u2013Senior

Question 3

"What is the empty-path parent route pattern and when would you use it?"

Strong answer covers:

- A route with `path: ''` in the parent adds nothing to the URL but can carry a layout component, guards, and providers
- Used to wrap multiple routes under a shared layout without adding a URL segment
- Common use case: separate public layout (marketing) from authenticated app layout using two empty-path parents with different layout components and guards
- Also used to scope services via `providers: []` to a group of routes without affecting the URL
- The `children` of an empty-path parent are resolved relative to the parent's parent path \u2014 the URL structure stays clean

Weak answer misses: The service-scoping use case \u2014 most candidates know it for layouts but don't know it's also the idiomatic way to scope DI providers to a group of routes.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Nested routes place a `children` array inside a parent route, with child paths relative to the parent path. The parent's component must contain a `<router-outlet>` \u2014 child components render there while the parent persists. This is the foundation of layout components, persistent shells, and feature-level route organisation.

Syntax at a glance:

Task	How
Define child routes	`children: [{ path: 'users', component: ... }]` inside parent route
Relative child path	`path: 'users'` not `path: '/users'` \u2014 no leading slash
Layout component template	`<router-outlet />` + `imports: [RouterOutlet]`
Relative `routerLink` in layout	`routerLink="users"` or `[routerLink]="['users']"`
Redirect on empty child path	`{ path: '', redirectTo: 'dashboard', pathMatch: 'full' }`
Read own route param	`inject(ActivatedRoute).paramMap.pipe(map(p => p.get('id')))`
Read parent route param	`inject(ActivatedRoute).parent!.paramMap.pipe(map(p => p.get('id')))`
Layout-only parent (no URL segment)	`{ path: '', component: LayoutComponent, children: [...] }`
Guard all children at once	`canActivate: [authGuard]` or `canActivateChild: [authGuard]` on parent
Lazy-load feature + layout	`loadChildren` on parent in `app.routes.ts`, `loadComponent` on layout in feature routes

When to use / when not to use:

Use nested routes whenever two or more pages share a persistent UI shell (sidebar, header, tabs). Use the empty-path parent pattern when you need a layout or guard without adding a URL segment. Do not nest routes just because components are related conceptually \u2014 only nest when the parent genuinely has a `<router-outlet>` that child components render inside.

The one rule that matters most:

The parent route's component must have `<router-outlet />` in its template and `RouterOutlet` in its `imports` \u2014 without this, child routes are configured correctly but silently render nothing.

HTTPCLIENT & API INTEGRATION

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The basic GET/POST calls are straightforward, but understanding typed responses, error handling, request configuration, and the interplay with RxJS operators is what separates production-quality HTTP code from tutorial-level code.

Angular Relevance: Critical \u2014 Almost every Angular app communicates with a backend. HttpClient is the official Angular mechanism for all HTTP communication. It is RxJS-native, interceptor-aware, and testable by design \u2014 understanding it deeply is non-negotiable for professional Angular development.

Section 1 \u2014 Explanation

The Postal Service Analogy

Imagine your Angular component is an office worker who needs to send and receive parcels from a warehouse (your API server).

HttpClient is the postal service. You hand it a parcel (a request \u2014 URL, method, body, headers) and it delivers the response back to you. You don't manage the trucks, the routing, or the network \u2014 that's the postal service's job.

But here's the key: the postal service doesn't hand you the parcel directly. It hands you a tracking slip \u2014 an Observable. The parcel (the HTTP response) hasn't arrived yet. You subscribe to the tracking slip, and when the parcel arrives, your callback runs.

This is why HttpClient methods return Observables, not values. The HTTP call hasn't happened yet when the Observable is created \u2014 it only starts when something subscribes.

Three important rules about this postal service:

- **No subscription = no request.** The Observable is cold \u2014 it only fires when subscribed.
- **Each subscription = a new request.** Subscribing twice sends two HTTP calls.
- **Unsubscribing = cancelling the parcel.** If you unsubscribe before the response arrives, Angular cancels the in-flight request.

Technical Explanation

HttpClient is an injectable Angular service (`@angular/common/http`). Every method returns a cold Observable that, when subscribed, fires an HTTP request and emits the response.

Method	HTTP Verb	When to Use
`http.get<T>(url, options?)`	GET	Fetch data

Method	HTTP Verb	When to Use
`http.post<T>(url, body, options?)`	POST	Create a resource
`http.put<T>(url, body, options?)`	PUT	Replace a resource fully
`http.patch<T>(url, body, options?)`	PATCH	Partially update a resource
`http.delete<T>(url, options?)`	DELETE	Remove a resource
`http.request<T>(method, url, options?)`	Any	Full control

The generic `<T>` tells TypeScript what shape the response body will be \u2014 Angular does not validate this at runtime; it's a compile-time cast.

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular registers HttpClient via `provideHttpClient()` in `app.config.ts` \u2014 no `HttpClientModule` needed. Interceptors use the functional pattern via `withInterceptors([])`. HttpClient is always injected via `inject()` in services.

Migration table:

Legacy Pattern	Modern Replacement
`HttpClientModule` in `AppModule` imports	`provideHttpClient()` in `app.config.ts`
`withInterceptorsFromDi()` + `HTTP_INTERCEPTORS` token	`provideHttpClient(withInterceptors([fn1, fn2]))`
Constructor injection of `HttpClient`	`private http = inject(HttpClient)`
Untyped `http.get(url)`	Typed `http.get<User[]>(url)`
Manual `unsubscribe()` in `ngOnDestroy`	`takeUntilDestroyed(destroyRef)`

Section 3 \u2014 Code Example

Plain concept \u2014 cold Observable, no request without subscription

```
// Creating the Observable does NOT fire the request
const request$ = http.get('https://api.example.com/users');

// The request fires HERE \u2014 when something subscribes
request$.subscribe(data => console.log(data));

// Subscribing AGAIN fires a SECOND independent request
request$.subscribe(data => console.log('second call:', data));
```

`app.config.ts` \u2014 registering HttpClient

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideRouter } from '@angular/router';
import { APP_ROUTES } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(APP_ROUTES),
    provideHttpClient(
      // Register functional interceptors here (Topic: HTTP Interceptors)
      // withInterceptors([authInterceptor, errorInterceptor])
      // Leave empty if no interceptors yet \u2014 provideHttpClient() alone is enough
    ),
  ],
};
```

`services/user.service.ts` \u2014 the correct architecture

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams, HttpErrorResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { map, catchError } from 'rxjs/operators';

// Always define the shape of API responses with interfaces
export interface User {
  id:    number;
  name:  string;
  email: string;
  role:  'admin' | 'user';
}

export interface PaginatedResponse<T> {
  data:     T[];
  total:    number;
  page:     number;
  pageSize: number;
}

@Injectable({ providedIn: 'root' })
export class UserService {
  private http    = inject(HttpClient);
  private baseUrl = 'https://api.example.com';

  // GET \u2014 list with query params
  getUsers(page = 1, pageSize = 20): Observable<PaginatedResponse<User>> {
    // HttpParams builds the query string immutably: ?page=1&pageSize=20
    const params = new HttpParams()
      .set('page',     page)
      .set('pageSize', pageSize);

    return this.http
      .get<PaginatedResponse<User>>(`${this.baseUrl}/users`, { params })
      .pipe(
        // Transform if the API shape doesn't match your model
        map(response => ({
          ...response,
```

```
        data: response.data.map(u => ({ ...u, name: u.name.trim() }))
      })),
      catchError(this.handleError)
    );
}

// GET \u2014 single resource
getUserById(id: number): Observable<User> {
  return this.http
    .get<User>(`${this.baseUrl}/users/${id}`)
    .pipe(catchError(this.handleError));
}

// POST \u2014 create
createUser(payload: Omit<User, 'id'>): Observable<User> {
  // Angular sets Content-Type: application/json automatically for object bodies
  return this.http
    .post<User>(`${this.baseUrl}/users`, payload)
    .pipe(catchError(this.handleError));
}

// PUT \u2014 full replace
updateUser(id: number, payload: User): Observable<User> {
  return this.http
    .put<User>(`${this.baseUrl}/users/${id}`, payload)
    .pipe(catchError(this.handleError));
}

// PATCH \u2014 partial update
patchUser(id: number, changes: Partial<User>): Observable<User> {
  return this.http
    .patch<User>(`${this.baseUrl}/users/${id}`, changes)
    .pipe(catchError(this.handleError));
}

// DELETE \u2014 often returns no body
deleteUser(id: number): Observable<void> {
  return this.http
    .delete<void>(`${this.baseUrl}/users/${id}`)
    .pipe(catchError(this.handleError));
}

// Centralised error handler \u2014 translates HTTP errors to domain errors
// Components never see raw HttpErrorResponse \u2014 only user-facing messages
private handleError(error: HttpErrorResponse): Observable<never> {
  let userMessage = 'An unexpected error occurred';

  if (error.status === 0) {
    userMessage = 'Network error \u2014 please check your connection';
  } else if (error.status === 401) {
    userMessage = 'Unauthorised \u2014 please log in again';
  } else if (error.status === 404) {
    userMessage = 'Resource not found';
  } else if (error.status >= 500) {
    userMessage = 'Server error \u2014 please try again later';
  }

  console.error('HTTP Error:', error);
  return throwError(() => new Error(userMessage));
```

```
  }
}
```

Component \u2014 consuming the service with loading and error states

```
import { Component, OnInit, inject, DestroyRef } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { UserService, User } from './services/user.service';

@Component({
  selector: 'app-users-list',
  standalone: true,
  template: `
    @if (loading) {
      <p>Loading users\u2026</p>
    } @else if (error) {
      <p class="error">{{ error }}</p>
      <button (click)="loadUsers()">Retry</button>
    } @else {
      <ul>
        @for (user of users; track user.id) {
          <li>{{ user.name }} \u2014 {{ user.email }}</li>
        }
      </ul>
    }
  `,
})
export class UsersListComponent implements OnInit {
  private userService = inject(UserService);
  private destroyRef  = inject(DestroyRef); // inject explicitly \u2014 works in any method

  users:   User[]        = [];
  loading: boolean       = false;
  error:   string | null = null;

  ngOnInit(): void { this.loadUsers(); }

  loadUsers(): void {
    this.loading = true;
    this.error   = null;

    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef)) // \u2705 DestroyRef passed explicitly
      .subscribe({
        next:  response => { this.users = response.data; this.loading = false; },
        error: err      => { this.error = err.message;  this.loading = false; },
      });
  }
}
```

Search with debounce and request cancellation

```
@Component({
  selector: 'app-search',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <input [formControl]="searchControl" placeholder="Search users\u2026" />
```

```
    @if (loading) { <span>Searching\u2026</span> }
    @for (user of results; track user.id) { <li>{{ user.name }}</li> }
    @if (error) { <p class="error">{{ error }}</p> }
  `,
})
export class SearchComponent {
  private userService = inject(UserService);

  searchControl = new FormControl('');
  results: User[]    = [];
  loading            = false;
  error: string | null = null;

  constructor() {
    this.searchControl.valueChanges.pipe(
      debounceTime(300),                              // wait 300ms after last keystroke
      distinctUntilChanged(),                         // skip if value unchanged
      filter(term => (term?.length ?? 0) >= 2),       // skip short queries
      tap(() => { this.loading = true; this.error = null; }),
      switchMap(term =>                               // cancel previous, start new
        this.userService.searchUsers(term ?? '').pipe(
          catchError(err => {
            this.error = err.message;
            return of([]);                            // recover \u2014 don't kill outer stream
          })
        )
      ),
      takeUntilDestroyed()                            // constructor context \u2014 no DestroyRef neede
    ).subscribe(results => {
      this.results = results;
      this.loading = false;
    });
  }
}
```

Parallel requests with `forkJoin`

```
export interface DashboardData {
  user:          User;
  orders:        Order[];
  notifications: Notification[];
}

@Injectable({ providedIn: 'root' })
export class DashboardService {
  private http = inject(HttpClient);

  loadDashboard(userId: number): Observable<DashboardData> {
    // forkJoin fires all three simultaneously \u2014 emits once when ALL complete
    // \u26a0\ufe0f forkJoin cancels all if ANY errors \u2014 add catchError per source for resilience
    return forkJoin({
      user:          this.http.get<User>(`/api/users/${userId}`),
      orders:        this.http.get<Order[]>(`/api/users/${userId}/orders`),
      notifications: this.http.get<Notification[]>(`/api/users/${userId}/notifications`),
    });
  }
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 HTTP in the component, untyped, no error handling

```
@Component({ ... })
export class BadComponent implements OnInit {
  private http = inject(HttpClient); // \u274c HttpClient in component \u2014 not reusable
  users: any[] = [];                 // \u274c any \u2014 loses all type safety

  ngOnInit() {
    // \u274c No error handling \u2014 404/500 silently kills the Observable
    // \u274c No loading state \u2014 blank screen until data arrives
    // \u274c No cleanup \u2014 callback may run on a destroyed component
    this.http.get('https://api.example.com/users').subscribe((data: any) => {
      this.users = data; // \u274c assumes response is directly an array \u2014 fragile
    });
  }
}
```

What breaks: API errors crash silently \u2014 the user sees nothing. No loading state means a blank screen on slow connections. HTTP logic is locked to one component \u2014 can't be reused or mocked in tests. `any` types mean TypeScript can't catch property name typos or shape mismatches. Potential memory leak if the component destroys mid-flight.

\u2705 After \u2014 typed service, error handling, loading state, cleanup

```
@Component({ ... })
export class GoodComponent implements OnInit {
  private userService = inject(UserService); // \u2705 consume the service
  private destroyRef  = inject(DestroyRef);

  users:   User[]        = [];
  loading              = false;
  error:   string | null = null;

  ngOnInit() { this.loadUsers(); }

  loadUsers() {
    this.loading = true;
    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next:  res => { this.users = res.data; this.loading = false; },
        error: err => { this.error = err.message; this.loading = false; },
      });
  }
}
```

Why this matters: The service is the boundary between HTTP concerns and UI concerns. Components shouldn't know about URLs, HTTP verbs, or error codes \u2014 they should only know "I asked for users and either got them or got an error." This separation makes the service independently testable and reusable across multiple components.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Not subscribing and wondering why nothing happens

What it is: `this.userService.createUser(this.formData)` \u2014 calling the service method but forgetting `.subscribe()`.

Why it happens: Coming from Promises, where calling `fetch(...)` immediately fires the request. With Observables, the call is lazy \u2014 nothing fires until subscription.

How to avoid: Always subscribe to HttpClient Observables, or use `async` pipe / `toSignal()` to subscribe automatically. If you want fire-and-forget, you still need `.subscribe()` \u2014 at minimum with an error handler.

Angular consequence: The API call silently never happens. No error, no feedback. This is the most common "my form submission does nothing" bug.

Mistake 2 \u2014 Putting HttpClient calls directly in components

What it is: Injecting `HttpClient` into a component and calling `this.http.get(...)` directly inside it.

Why it happens: Faster to write \u2014 fewer files, fewer injections. Tutorials often do this to keep examples short.

How to avoid: Always create a dedicated service for each API domain (`UserService`, `ProductService`). Inject `HttpClient` only in services.

Angular consequence: HTTP logic is locked to one component \u2014 can't be reused, can't be mocked in tests, and interceptors can't be applied cleanly. At scale, duplicated API logic appears across multiple components.

Mistake 3 \u2014 Ignoring the error callback in `.subscribe()`

What it is: `.subscribe(data => { this.users = data; })` with no error handler.

Why it happens: The happy path works perfectly in development where APIs are stable. Error handling feels optional until production.

How to avoid: Always provide an error callback: `.subscribe({ next, error })`. At minimum, log the error and set a user-facing error state.

Angular consequence: In production, API failures leave the component in a permanent loading or empty state. The user has no way to know what happened or retry.

Mistake 4 \u2014 `takeUntilDestroyed()` called outside injection context

What it is: Using `takeUntilDestroyed()` without arguments inside `ngOnInit()` or a method, triggering `Error: takeUntilDestroyed() can only be used within an injection context`.

Why it happens: The no-argument form works beautifully in the constructor or field initializers \u2014 developers assume it works everywhere.

How to avoid: Inject `DestroyRef` explicitly as a field: `private destroyRef = inject(DestroyRef)`. Then pass it anywhere: `takeUntilDestroyed(this.destroyRef)`.

Angular consequence: Runtime error thrown the moment `loadUsers()` is called. The component never loads data.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Inject `HttpClient` in services \u2014 never directly in components	Call `http.get()` inside a component class
Type every response with an interface: `http.get<User[]>(url)`	Use `any` as the response type
Always handle errors \u2014 `catchError` in the service, `error` callback in the component	Provide only a `next` callback and ignore errors
Use `HttpParams` to build query strings	Manually concatenate query strings: `url + '?page=' + page`
Use `takeUntilDestroyed(destroyRef)` for all component HTTP subscriptions	Let HTTP subscriptions run without cleanup
Use `provideHttpClient()` in `app.config.ts`	Use `HttpClientModule` \u2014 legacy NgModule pattern
Use `switchMap` for user-triggered searches \u2014 cancels previous in-flight request	Use `mergeMap` for search \u2014 allows stale responses to overwrite current results
Use `forkJoin` for parallel independent requests	Nest `.subscribe()` inside `.subscribe()` for sequential calls
Translate HTTP errors to domain errors in the service	Expose raw `HttpErrorResponse` to the component

---

Section 7 \u2014 Interview Prep

Question 1

"Why does Angular's HttpClient return an Observable instead of a Promise, and what practical advantages does that give you?"

Strong answer covers:

- Observables are lazy \u2014 no request fires until subscription; Promises execute immediately on creation

- Observables are cancellable \u2014 unsubscribing from an HttpClient Observable actually cancels the in-flight XHR request in the browser; Promises cannot be cancelled
- Observables are composable \u2014 `pipe(debounceTime, switchMap, retry, catchError)` chains are impossible with Promises
- `switchMap` enables automatic cancellation of previous requests when new ones arrive \u2014 critical for search
- `takeUntilDestroyed` cleans up HTTP subscriptions automatically when a component destroys
- `firstValueFrom()` bridges to Promises when the calling context genuinely expects one (resolvers, guards)

Weak answer misses: The cancellation point \u2014 most candidates mention composability but miss that unsubscribing from an HttpClient Observable actually cancels the XHR request in the browser, not just ignores the response.

Level: Mid

Question 2

"In a search input that calls an API on every keystroke, how would you prevent excessive API calls and handle out-of-order responses?"

Strong answer covers:

- `debounceTime(300)` \u2014 waits 300ms after the last keystroke before emitting
- `distinctUntilChanged()` \u2014 skips emission if the value hasn't changed
- `filter` \u2014 skips very short queries (under 2 characters)
- `switchMap` \u2014 cancels the previous in-flight request when a new value arrives; only the latest request's response reaches the subscriber
- Why `mergeMap` would be wrong \u2014 it doesn't cancel, so out-of-order responses cause race conditions where stale results overwrite current ones silently

Weak answer misses: The distinction between `switchMap` and `mergeMap` \u2014 specifically that `switchMap` cancels the previous Observable, which is what prevents stale responses from rendering.

Level: Mid\u2013Senior

Question 3

"How do you handle HTTP errors in Angular, and where in the architecture should error handling live?"

Strong answer covers:

- Two layers: `catchError` in the service (translates HTTP errors to domain errors), `error` callback in the component (updates UI state)
- `HttpErrorResponse` has `status` (numeric code), `message`, `error` (body), and `status === 0` for network errors
- `throwError(() => new Error(message))` re-throws as an Observable error for downstream handlers
- The service should never expose raw `HttpErrorResponse` to the component \u2014 translate to user-facing messages
- Global cross-cutting concerns (401 redirect to login, logging) belong in HTTP Interceptors

Weak answer misses: The two-layer architecture \u2014 service translates, component displays. Candidates who only mention one layer miss the separation of concerns that makes Angular code maintainable at scale.

Level: Mid\u2013Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

HttpClient is an injectable Angular service that wraps every HTTP request as a cold Observable \u2014 nothing fires until subscription. Every method is typed via generics but Angular does not validate the shape at runtime. All HTTP calls belong in services, all errors belong in `catchError`, and all component subscriptions need `takeUntilDestroyed`.

Syntax at a glance:

Task	How
GET with query params	`http.get<T>(url, { params: new HttpParams().set('k', v) })`
POST with JSON body	`http.post<T>(url, body)` \u2014 Angular sets `Content-Type: application/json` automatically
Set custom headers	`http.get<T>(url, { headers: new HttpHeaders({ 'X-Custom': 'value' }) })`
GET full response (status + headers)	`http.get<T>(url, { observe: 'response' })` \u2192 `HttpResponse<T>`
Parallel requests	`forkJoin({ a: obs1, b: obs2 })`
Sequential requests	`obs1.pipe(switchMap(result => obs2(result)))`
Convert to Promise	`firstValueFrom(http.get<T>(url))`
Cleanup in component	`takeUntilDestroyed(this.destroyRef)`
Register HttpClient	`provideHttpClient()` in `app.config.ts`

Choosing the right operator for HTTP composition:

Scenario	Operator	Why
Sequential calls (result of A feeds B)	`switchMap`	Chains and cancels previous
Parallel independent calls	`forkJoin`	Fires all at once, waits for all
User search (cancel previous)	`switchMap`	Cancels in-flight stale request
Upload queue (preserve order)	`concatMap`	Queues sequentially
Retry on failure	`retry(3)`	Re-subscribes up to N times

When to use / when not to use:

Use `HttpClient` for all HTTP communication \u2014 it is the only Angular-native mechanism. Use `forkJoin` when multiple requests are independent and all results are needed before rendering. Use `switchMap` for any user-triggered request that could fire repeatedly \u2014 search, tab changes, param changes. Do not use `mergeMap` for search \u2014 it allows race conditions. Do not use `firstValueFrom` as a wholesale replacement for Observables in components \u2014 it loses cancellation.

The one rule that matters most:

HttpClient belongs in services, responses belong in typed interfaces, errors belong in `catchError`, and subscriptions belong in `takeUntilDestroyed(destroyRef)` \u2014 every omission of these four is a production bug waiting to happen.

HTTP INTERCEPTORS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The concept is straightforward once you understand middleware pipelines, but the Angular-specific setup (functional vs class-based, `provideHttpClient` configuration, cloning requests) has enough nuance to trip up developers who learn the pattern without understanding the immutability requirement.

Angular Relevance: Critical \u2014 Interceptors are the production solution for cross-cutting HTTP concerns: auth tokens, logging, error handling, loading indicators, and request retries. Without them, these concerns get duplicated across every service. Every professional Angular app uses at least one interceptor.

Section 1 \u2014 Explanation

The Airport Security Analogy

Imagine every HTTP request your app makes is a passenger trying to board a flight, and every response is a passenger returning from a trip.

Without an interceptor: each passenger handles their own security check, their own customs declaration, their own baggage tagging \u2014 separately, every single time.

With an interceptor: you install a security checkpoint that every passenger passes through automatically. The checkpoint can add something to every outbound bag (attach an auth token to every request), inspect every inbound bag (log every response), stop a bag and reroute it (retry on 401, redirect to login on 403), or count bags passing through (show a global loading spinner).

The passengers (your services) don't know the checkpoint exists. They just hand their bag to the airline (HttpClient) and trust it arrives. The interceptor is invisible middleware that every request and response passes through automatically.

Technical Explanation

An interceptor sits between `HttpClient` and the actual network. When `http.get(url)` is called, Angular passes the request through the interceptor chain before sending it. When the response arrives, it passes back through the chain in reverse.

```
HttpClient call
    \u2193
Interceptor 1 (outbound)
    \u2193
Interceptor 2 (outbound)
    \u2193
    Network (actual HTTP request)
    \u2193
Interceptor 2 (inbound \u2014 response)
```

```
    \u2193
Interceptor 1 (inbound \u2014 response)
    \u2193
Subscriber (your component/service)
```

Modern Angular (v15+) uses **functional interceptors** \u2014 plain functions that are simpler, tree-shakeable, and easier to test than the older class-based `HttpInterceptor` interface.

Functional Interceptor (modern)	Class-based Interceptor (legacy)	
Syntax	`HttpInterceptorFn` \u2014 a plain function	`implements HttpInterceptor` class
Registration	`provideHttpClient(withInterceptors([...]))`	`HTTP_INTERCEPTORS` multi-provider token
Tree-shaking	\u2705 Yes	\u274c No
DI inside interceptor	`inject()` at call time	Constructor injection
Angular version	v15+ preferred	All versions \u2014 deprecated

The immutability rule: `HttpRequest` objects are immutable. You cannot modify them directly. To change a request (add a header, change a URL), you must call `request.clone({ ... })` to get a new request with your modifications, then pass the cloned version to `next`.

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular interceptors are plain functions typed as `HttpInterceptorFn`. They are registered via `provideHttpClient(withInterceptors([...]))` in `app.config.ts`. The class-based `implements HttpInterceptor` pattern is deprecated \u2014 never use it in new code.

Migration table:

Legacy Pattern	Modern Replacement
`@Injectable() class AuthInterceptor implements HttpInterceptor`	`export const authInterceptor: HttpInterceptorFn = (req, next) => ...`
`HTTP_INTERCEPTORS` multi-provider token	`provideHttpClient(withInterceptors([authInterceptor]))`
`withInterceptorsFromDi()`	Remove entirely \u2014 only exists to bridge legacy class-based interceptors
Constructor injection inside guard class	`inject()` at the top of the interceptor function body
`next.handle(req)`	`next(req)`

Section 3 \u2014 Code Example

Plain concept \u2014 the interceptor contract

```
import { HttpInterceptorFn, HttpRequest, HttpHandlerFn, HttpEvent } from '@angular/common/http';
import { Observable } from 'rxjs';

// A functional interceptor is just a function matching HttpInterceptorFn
// req:  the outbound request (immutable)
// next: passes the request to the next interceptor or the network
// Returns: Observable<HttpEvent<unknown>> \u2014 the response stream

const myInterceptor: HttpInterceptorFn = (
  req: HttpRequest<unknown>,
  next: HttpHandlerFn
): Observable<HttpEvent<unknown>> => {

  // 1. Clone to modify \u2014 never mutate req directly
  const modifiedReq = req.clone({
    headers: req.headers.set('X-Custom-Header', 'hello')
  });

  // 2. Pass to next and return the response Observable
  return next(modifiedReq);

  // 3. Optionally pipe operators onto the response
  // return next(modifiedReq).pipe(
  //   tap(event => console.log('Response:', event))
  // );
};
```

`interceptors/auth.interceptor.ts` **\u2014 attaching a Bearer token**

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../services/auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService); // inject() works inside interceptor functions
  const token       = authService.getToken();

  // Skip if no token \u2014 user is not logged in
  if (!token) return next(req);

  // Only attach to your own API \u2014 never send tokens to third-party URLs
  const isOwnApi = req.url.startsWith('https://api.myapp.com');
  if (!isOwnApi) return next(req);

  // Clone \u2014 HttpRequest is immutable, never mutate directly
  const authenticatedReq = req.clone({
    headers: req.headers.set('Authorization', `Bearer ${token}`)
  });

  return next(authenticatedReq);
};
```

`interceptors/error.interceptor.ts` **\u2014 global error handling**

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject }    from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { Router }    from '@angular/router';
```

```
import { ToastService } from '../services/toast.service';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router       = inject(Router);
  const toastService = inject(ToastService);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      const isLoginRequest = req.url.includes('/auth/login');
      const isLoginPage    = router.url.includes('/login');

      if (error.status === 401 && !isLoginRequest && !isLoginPage) {
        // Exclude login endpoint \u2014 a failed login legitimately returns 401
        // Redirect loop prevention: don't redirect if already on login page
        router.navigate(['/login'], { queryParams: { returnUrl: router.url } });
      }

      if (error.status >= 500) {
        toastService.show('Server error \u2014 please try again later', 'error');
      }

      // Always re-throw \u2014 downstream catchError and component error callbacks must still run
      return throwError(() => error);
    })
  );
};
```

`interceptors/loading.interceptor.ts` \u2014 global loading spinner

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { finalize } from 'rxjs/operators';
import { LoadingService } from '../services/loading.service';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);

  loadingService.increment(); // add one to the in-flight request count

  return next(req).pipe(
    // finalize runs on BOTH complete AND error \u2014 unlike tap(complete) which only runs on complete
    finalize(() => loadingService.decrement())
  );
};
```

`services/loading.service.ts` \u2014 signal-based counter

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class LoadingService {
  private requestCount = signal(0);

  // computed() \u2014 true when any request is in-flight
  // Counter (not boolean) handles concurrent requests correctly:
  // spinner stays until ALL requests complete, not just the first one
  isLoading = computed(() => this.requestCount() > 0);

  increment(): void { this.requestCount.update(n => n + 1); }
```

```
  decrement(): void { this.requestCount.update(n => Math.max(0, n - 1)); }
}
```

`interceptors/logging.interceptor.ts` \u2014 request/response timing

```
import { HttpInterceptorFn, HttpResponse } from '@angular/common/http';
import { tap } from 'rxjs/operators';

export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const startTime = Date.now();

  return next(req).pipe(
    tap({
      next: event => {
        if (event instanceof HttpResponse) {
          const elapsed = Date.now() - startTime;
          console.log(`[HTTP] ${req.method} ${req.url} \u2192 ${event.status} (${elapsed}ms)`);
        }
      },
      error: error => {
        const elapsed = Date.now() - startTime;
        console.error(`[HTTP ERROR] ${req.method} ${req.url} \u2192 ${error.status} (${elapsed}ms)`);
      }
    })
  );
};
```

`app.config.ts` \u2014 registering interceptors in order

```
import { ApplicationConfig }  from '@angular/core';
import { provideRouter }      from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor }    from './interceptors/auth.interceptor';
import { errorInterceptor }   from './interceptors/error.interceptor';
import { loadingInterceptor } from './interceptors/loading.interceptor';
import { loggingInterceptor } from './interceptors/logging.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(APP_ROUTES),
    provideHttpClient(
      withInterceptors([
        // Order matters: top-to-bottom on outbound requests,
        // bottom-to-top on inbound responses
        loggingInterceptor,   // outermost \u2014 sees everything first and last
        loadingInterceptor,   // second
        authInterceptor,      // third \u2014 attaches token before sending
        errorInterceptor,     // innermost \u2014 closest to the actual response
      ])
    ),
  ],
};
```

`interceptors/token-refresh.interceptor.ts` \u2014 silent token refresh

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject }        from '@angular/core';
import { catchError, switchMap, throwError } from 'rxjs';
```

```
import { AuthService }   from '../services/auth.service';

export const tokenRefreshInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      // Only attempt refresh on 401 \u2014 and never for the refresh call itself
      // (prevents infinite loop if the refresh endpoint also returns 401)
      const isRefreshRequest = req.url.includes('/auth/refresh');
      if (error.status !== 401 || isRefreshRequest) {
        return throwError(() => error);
      }

      return authService.refreshToken().pipe(
        switchMap(newToken => {
          // Retry the original request with the new token
          const retryReq = req.clone({
            headers: req.headers.set('Authorization', `Bearer ${newToken}`)
          });
          return next(retryReq);
        }),
        catchError(refreshError => {
          // Refresh failed \u2014 log out and redirect
          authService.logout();
          return throwError(() => refreshError);
        })
      );
    })
  );
};
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 auth token duplicated in every service

```
@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers() {
    const token   = localStorage.getItem('auth_token'); // \u274c duplicated
    const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
    return this.http.get<User[]>('/api/users', { headers });
  }

  deleteUser(id: number) {
    const token   = localStorage.getItem('auth_token'); // \u274c duplicated again
    const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
    return this.http.delete(`/api/users/${id}`, { headers });
  }
}

@Injectable({ providedIn: 'root' })
export class OrderService {
  getOrders() {
    const token   = localStorage.getItem('auth_token'); // \u274c and again
    const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
    return this.http.get<Order[]>('/api/orders', { headers });
```

```
  }
}
```

What breaks: Token retrieval is duplicated across every service method. If the auth scheme changes (`Bearer` \u2192 `API-Key`), you update dozens of files. If you forget the header in one place, that endpoint silently fails auth. Adding retry logic, logging, or global error handling uniformly is impossible.

\u2705 After \u2014 auth handled once in an interceptor

```
// authInterceptor handles all requests automatically \u2014 services are completely clean

@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers()            { return this.http.get<User[]>('/api/users'); }
  deleteUser(id:number) { return this.http.delete(`/api/users/${id}`); }
}

@Injectable({ providedIn: 'root' })
export class OrderService {
  getOrders() { return this.http.get<Order[]>('/api/orders'); }
}
// Token is attached by authInterceptor \u2014 invisible to every service
```

Why this matters: The interceptor is a single point of change. Update the auth scheme in one file \u2014 every request in the entire app is updated instantly. Services contain only domain logic, no infrastructure concerns.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Mutating the request instead of cloning it

What it is: `req.headers.set('Authorization', 'Bearer token')` called directly, then passing the original `req` to `next`.

Why it happens: `req.headers.set(...)` looks like a mutation. But `HttpHeaders` is immutable \u2014 `.set()` returns a new `HttpHeaders` object; it does not modify the existing one. The original `req` is passed unchanged.

How to avoid: Always use `req.clone({ headers: req.headers.set(...) })`. The `.clone()` method is the only correct way to produce a modified request.

Angular consequence: The header is silently not added. The API returns 401. The developer debugs the auth service for hours when the bug is in the interceptor \u2014 no error, no warning, just missing headers.

Mistake 2 \u2014 Sending auth tokens to third-party URLs

What it is: The auth interceptor attaches the Bearer token to every URL unconditionally \u2014 including CDNs, analytics endpoints, and third-party APIs the app calls.

Why it happens: The interceptor is meant to be universal. Developers don't consider that the app might call external services alongside their own API.

How to avoid: Always check the URL before attaching credentials:

```
const isOwnApi = req.url.startsWith('https://api.myapp.com');
if (!isOwnApi) return next(req);
```

Angular consequence: Your private auth token is leaked to third-party servers in request headers \u2014 a security vulnerability that is invisible in testing and only discoverable via network inspection in production.

Mistake 3 \u2014 Using `tap(complete)` instead of `finalize()` for cleanup

What it is: Using `tap({ complete: () => loadingService.decrement() })` for loading state cleanup.

Why it happens: `tap` is the standard side-effect operator \u2014 developers reach for it instinctively. The `complete` callback feels like the right place to clean up.

How to avoid: Use `finalize(() => loadingService.decrement())`. `finalize` runs on both `complete` and `error`. The `tap(complete)` callback does not run when an Observable errors \u2014 only when it completes successfully.

Angular consequence: The loading spinner stays on screen permanently after any HTTP error. The user sees a frozen spinner with no way to dismiss it.

Mistake 4 \u2014 Registering interceptors in the wrong order

What it is: Registering `authInterceptor` after `errorInterceptor`, expecting auth to run first on outbound requests.

Why it happens: Order isn't obvious. Developers assume last-registered means highest priority (like some middleware frameworks), or add interceptors without understanding the pipeline direction.

How to avoid: Interceptors run top-to-bottom on outbound requests and bottom-to-top on inbound responses in the order they appear in `withInterceptors([...])`. The first in the array is the outermost wrapper.

Angular consequence: Auth tokens not attached before error handling runs. Logging interceptors placed after auth see requests without the token \u2014 logging incomplete data.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Always `req.clone({...})` to modify a request	Mutate `req` directly or call `req.headers.set(...)` without cloning
Check `req.url` before attaching auth tokens \u2014 only send to your own API	Send auth headers to every URL including third-party services
Use `finalize()` for cleanup \u2014 it runs on both complete and error	Use `tap(complete)` for cleanup \u2014 it doesn't run on error
Use `inject()` inside the interceptor function body	Try to add constructor injection to functional interceptors
Register with `provideHttpClient(withInterceptors([...]))`	Register functional interceptors using the legacy `HTTP_INTERCEPTORS` token
Always re-throw errors with `throwError()` after handling	Swallow errors with `EMPTY` or `of(null)` \u2014 kills all downstream error handlers
Exclude auth/refresh endpoints from 401 redirect logic	Apply 401 redirect to all URLs \u2014 causes infinite loops on login pages
Use a counter signal in `LoadingService` for concurrent request tracking	Use a boolean `isLoading` flag \u2014 breaks when multiple requests run simultaneously

---

Section 7 \u2014 Interview Prep

Question 1

"What is an HTTP interceptor in Angular, and what are the most common use cases for one?"

Strong answer covers:

- Middleware that intercepts every HTTP request/response in the application transparently \u2014 services and components are unaware
- Common uses: attaching auth tokens, global error handling (401 redirect), loading state management, logging request/response timing, base URL injection, token refresh
- Modern Angular uses functional interceptors (`HttpInterceptorFn`) registered via `withInterceptors()`
- Requests are immutable \u2014 must use `req.clone()` to modify
- Interceptors run in order: top-to-bottom on requests, bottom-to-top on responses

Weak answer misses: The immutability requirement \u2014 candidates who haven't used interceptors in production often don't know that `req.headers.set()` returns a new object and doesn't mutate `req`.

Level: Mid

Question 2

"How would you implement a global loading spinner that activates during any HTTP request and deactivates when it completes or errors?"

Strong answer covers:

- A `LoadingService` with a signal tracking in-flight request count (not a boolean)
- A loading interceptor that increments on request start and decrements in `finalize()`
- Why `finalize` is essential \u2014 it runs on both complete and error; `tap(complete)` does not run on error
- The counter approach handles concurrent requests correctly \u2014 spinner stays until all are done, not just the first
- The spinner component subscribes to `loadingService.isLoading` computed signal

Weak answer misses: The counter vs boolean distinction \u2014 using a simple `isLoading = true/false` boolean breaks when two requests run simultaneously and the first finishes before the second.

Level: Mid

Question 3

"Walk me through how you would implement silent token refresh \u2014 automatically refreshing an expired JWT and retrying the original request without the user noticing."

Strong answer covers:

- Intercept 401 responses in the error interceptor
- Call `authService.refreshToken()` to get a new token
- Use `switchMap` to take the new token and retry the original request with `req.clone({ headers: ... })`
- Loop prevention: check `req.url.includes('/auth/refresh')` before attempting refresh \u2014 a 401 on the refresh endpoint itself should trigger logout, not another refresh
- The concurrent refresh race condition: multiple in-flight requests can all 401 simultaneously, each independently triggering a refresh call \u2014 a production implementation uses a shared Observable with `shareReplay(1)` so only one refresh call is made
- Fall back to `authService.logout()` if the refresh fails

Weak answer misses: The concurrent refresh race condition \u2014 multiple in-flight requests can all 401 simultaneously. A production implementation shares a single refresh Observable to prevent multiple refresh API calls.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

An HTTP interceptor is an invisible middleware function that every request and response passes through automatically. It sits between `HttpClient` and the network \u2014 services and components are completely unaware it exists. Used for auth tokens, global error handling, loading state, logging, and token refresh.

Syntax at a glance:

Task	How
Add a request header	`req.clone({ headers: req.headers.set('key', 'value') })`
Replace the URL	`req.clone({ url: newUrl })`
Add query params	`req.clone({ params: req.params.set('key', 'value') })`
Inspect the response	`next(req).pipe(tap(event => ...))`
Handle errors globally	`next(req).pipe(catchError(err => ...))`
Run cleanup always	`next(req).pipe(finalize(() => ...))`
Skip interceptor for a URL	`if (req.url.includes('skip')) return next(req)`
Retry a request	`next(req).pipe(retry({ count: 2, delay: 1000 }))`
Register interceptors	`provideHttpClient(withInterceptors([fn1, fn2]))`

Interceptor order cheat sheet:

```
withInterceptors([
  loggingInterceptor,    // runs 1st on request, 4th on response
  loadingInterceptor,    // runs 2nd on request, 3rd on response
  authInterceptor,       // runs 3rd on request, 2nd on response
  errorInterceptor,      // runs 4th on request, 1st on response (closest to network)
])
```

Cross-cutting concern \u2192 correct tool:

Concern	Tool
Attach auth token	`authInterceptor` \u2014 clone + set header
Global error handling	`errorInterceptor` \u2014 `catchError` + `throwError`
Loading spinner	`loadingInterceptor` \u2014 counter signal + `finalize`
Request/response logging	`loggingInterceptor` \u2014 `tap` + `Date.now()`

Concern	Tool
Token refresh on 401	`tokenRefreshInterceptor` \u2014 `catchError` + `switchMap`
Prepend environment base URL	`baseUrlInterceptor` \u2014 check relative URL + clone

The one rule that matters most:

Always `req.clone()` \u2014 never mutate the request. Always `throwError()` after handling an error \u2014 never swallow it with `EMPTY` or `of(null)`, because doing so kills every downstream `catchError` and `error` callback in the entire app.

Next topic in order: Reactive Forms (Phase 5 \u2014 Data & Forms, Topic 39 in your plan).

REACTIVE FORMS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 Reactive Forms introduce a programmatic, model-driven approach that requires understanding class composition, Observables, and validation trees working in concert \u2014 more moving parts than most Angular topics to this point.

Angular Relevance: Critical \u2014 Reactive Forms are the standard for any non-trivial form in Angular. Login screens, multi-step wizards, dynamic field arrays, real-time validation \u2014 all of these live here. If you build Angular apps professionally, you will write Reactive Forms every day.

Section 1 \u2014 Explanation

The Blueprint Analogy

Imagine you're building a house. You have two approaches.

Approach A (Template-Driven): you tell the workers verbally what to build as they go. Simple shed? Fine. But if the structure is complex, verbal instructions get messy fast \u2014 you can't easily inspect the plan, test it, or change it programmatically.

Approach B (Reactive/Blueprint-Driven): you draw a precise blueprint before construction. Every room (`FormGroup`), every fixture (`FormControl`), every repeated unit like identical apartments (`FormArray`) is defined explicitly in the blueprint. You can inspect it, validate it, change it programmatically, and test it without ever touching the physical building.

Reactive Forms are the Blueprint Approach. The form model lives entirely in your TypeScript class \u2014 not in the template. The template merely connects to the model. This separation is what makes Reactive Forms testable, scalable, and predictable.

Technical Explanation

Reactive Forms are built from three core building blocks:

Building Block	Role	Analogy
`FormControl`	A single field	One input box (one room fixture)
`FormGroup`	A collection of controls	A group of related fields (one floor of the house)
`FormArray`	A dynamic list of controls or groups	Identical repeated apartments on a floor
`AbstractControl`	Base class for all three	The shared blueprint standard all three follow

The entire form is a tree of these three classes. At runtime, every control tracks its current value, its validity status (`valid`, `invalid`, `pending`), its dirty/pristine status (has the user typed anything?), its touched/untouched status (has the user left the field?), a `valueChanges` Observable emitting every time the value changes, and a `statusChanges` Observable emitting every time validity changes.

Because `valueChanges` is an Observable, you get the full power of RxJS \u2014 debouncing, switching, combining \u2014 wired directly into your form.

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Reactive Forms use `inject(FormBuilder)` instead of constructor injection, `takeUntilDestroyed()` for subscription cleanup, standalone components with `ReactiveFormsModule` in `imports`, and `@if`/`@for` for template control flow. Strictly typed `FormGroup<{...}>` is available from Angular v14+.

Migration table:

Legacy Pattern	Modern Replacement
Constructor injection of `FormBuilder`	`private fb = inject(FormBuilder)`
`NgModule` with `ReactiveFormsModule` in `imports`	`ReactiveFormsModule` in standalone component `imports` array
Manual `ngOnDestroy` + `subscription.unsubscribe()`	`takeUntilDestroyed()` in constructor
`*ngIf` for validation feedback	`@if (control.invalid &amp;&amp; control.touched)`
`*ngFor` over `FormArray.controls`	`@for (ctrl of formArray.controls; track $index)`
Untyped `FormGroup`	`FormGroup<{ email: FormControl<string> }>` (strict typing)

Section 3 \u2014 Code Example

Plain TS \u2014 core building blocks in isolation

```
import { FormControl, FormGroup, FormArray, Validators } from '@angular/forms';

// FormControl: one field
const emailControl = new FormControl('', [Validators.required, Validators.email]);
console.log(emailControl.value);  // ''
console.log(emailControl.valid);  // false

emailControl.setValue('user@example.com');
console.log(emailControl.valid);  // true
// FormGroup: related fields grouped together
```

```
const loginForm = new FormGroup({
  email:    new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', [Validators.required, Validators.minLength(8)])
});

console.log(loginForm.value); // { email: '', password: '' }
loginForm.patchValue({ email: 'user@example.com' }); // update one field \u2014 password untouched

// FormArray: dynamic list of controls
const tagsArray = new FormArray([
  new FormControl('angular'),
  new FormControl('typescript')
]);

tagsArray.push(new FormControl('rxjs')); // add dynamically
tagsArray.removeAt(0);                   // remove by index
console.log(tagsArray.value);            // ['typescript', 'rxjs']
```

`profile-form.component.ts` \u2014 complete working example

```
import { Component, inject } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, FormArray, Validators } from '@angular/forms';

interface ProfileFormValue {
  username: string;
  email:    string;
  skills:   string[];
}

@Component({
  selector: 'app-profile-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="profileForm" (ngSubmit)="onSubmit()">

      <input formControlName="username" placeholder="Username" />
      @if (username.invalid && username.touched) {
        <span class="error">Username is required (min 3 chars)</span>
      }

      <input formControlName="email" type="email" placeholder="Email" />
      @if (email.invalid && email.touched) {
        @if (email.errors?.['required']) { <span>Email is required</span> }
        @if (email.errors?.['email'])    { <span>Must be a valid email</span> }
      }

      <!-- FormArray binding: formArrayName on container, [formControlName]="$index" on inputs -->
      <div formArrayName="skills">
        @for (skill of skills.controls; track $index) {
          <div>
            <input [formControlName]="$index" placeholder="Skill" />
            <button type="button" (click)="removeSkill($index)">Remove</button>
          </div>
        }
      </div>
      <button type="button" (click)="addSkill()">+ Add Skill</button>

      <button type="submit" [disabled]="profileForm.invalid">Save</button>
```

```
    </form>
  `,
})
export class ProfileFormComponent {
  private fb = inject(FormBuilder);

  // FormBuilder shorthand \u2014 equivalent to new FormGroup({ ... new FormControl(...) })
  profileForm: FormGroup = this.fb.group({
    username: ['', [Validators.required, Validators.minLength(3)]],
    email:    ['', [Validators.required, Validators.email]],
    skills:   this.fb.array([this.fb.control('')]),
  });

  constructor() {
    // \u2705 Subscribe in constructor \u2014 takeUntilDestroyed() requires injection context
    this.profileForm.valueChanges
      .pipe(
        debounceTime(300),      // don't fire on every keystroke
        takeUntilDestroyed()    // auto-unsubscribes when component destroys
      )
      .subscribe(value => console.log('Form changed:', value));
  }

  // Convenience getters \u2014 avoids repeating profileForm.get('...') in the template
  get username() { return this.profileForm.get('username')!; }
  get email()    { return this.profileForm.get('email')!; }
  get skills()   { return this.profileForm.get('skills') as FormArray; }

  addSkill(): void {
    this.skills.push(this.fb.control('', Validators.required));
  }

  removeSkill(index: number): void {
    this.skills.removeAt(index);
  }

  onSubmit(): void {
    if (this.profileForm.invalid) {
      // markAllAsTouched reveals ALL validation errors \u2014 for keyboard/automation users
      // who submit without touching any field
      this.profileForm.markAllAsTouched();
      return;
    }
    const formValue = this.profileForm.value as ProfileFormValue;
    console.log('Submitted:', formValue);
  }
}
```

Registration form with cross-field validator

```
@Component({ standalone: true, imports: [ReactiveFormsModule], template: `...` })
export class RegisterComponent {
  private fb = inject(FormBuilder);

  registerForm = this.fb.group(
    {
      username:        ['', [Validators.required, Validators.minLength(3)]],
      email:           ['', [Validators.required, Validators.email]],
      password:        ['', [Validators.required, Validators.minLength(8)]],
```

```
      confirmPassword: ['', Validators.required],
    },
    { validators: this.passwordMatchValidator } // group-level \u2014 can see all sibling controls
  );

  get f() { return this.registerForm.controls; }

  // Cross-field validator lives on the FormGroup, not on a control
  // Error is on form.errors, not on confirmPassword.errors
  passwordMatchValidator(group: AbstractControl) {
    const pass    = group.get('password')?.value;
    const confirm = group.get('confirmPassword')?.value;
    return pass === confirm ? null : { passwordMismatch: true };
  }

  register() {
    if (this.registerForm.invalid) {
      this.registerForm.markAllAsTouched();
      return;
    }
    console.log(this.registerForm.value);
  }
}
```

Dynamic invoice with `FormArray` of `FormGroup`s

```
@Component({ standalone: true, imports: [ReactiveFormsModule], template: `
  <form [formGroup]="invoiceForm">
    <div formArrayName="lineItems">
      @for (item of lineItems.controls; track $index) {
        <div [formGroupName]="$index">
          <input formControlName="description" placeholder="Description" />
          <input formControlName="quantity" type="number" />
          <input formControlName="price" type="number" />
          <button type="button" (click)="removeItem($index)">\u2715</button>
        </div>
      }
    </div>
    <button type="button" (click)="addItem()">+ Add Item</button>
    <p>Total: {{ total | currency }}</p>
  </form>
`})
export class InvoiceComponent {
  private fb = inject(FormBuilder);

  invoiceForm = this.fb.group({
    lineItems: this.fb.array([this.createItem()])
  });

  get lineItems() { return this.invoiceForm.get('lineItems') as FormArray; }

  get total(): number {
    return this.lineItems.controls.reduce((sum, ctrl) => {
      const { quantity, price } = ctrl.value;
      return sum + (quantity * price || 0);
    }, 0);
  }

  createItem(): FormGroup {
```

```
    return this.fb.group({
      description: ['', Validators.required],
      quantity:    [1,  [Validators.required, Validators.min(1)]],
      price:       [0,  [Validators.required, Validators.min(0)]],
    });
  }

  addItem():             { this.lineItems.push(this.createItem()); }
  removeItem(i: number): { this.lineItems.removeAt(i); }
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 manual construction, no cleanup, no validators

```
export class OldFormComponent implements OnInit, OnDestroy {
  // Manually constructing every control \u2014 verbose, no validators
  loginForm = new FormGroup({
    email:    new FormControl(''), // \u274c no validators \u2014 always valid
    password: new FormControl('')
  });

  private sub: any; // \u274c typed as any

  ngOnInit() {
    // \u274c Subscribing without cleanup \u2014 memory leak on every component recreation
    this.sub = this.loginForm.valueChanges.subscribe(val => console.log(val));
  }

  onSubmit() {
    // \u274c No validity guard \u2014 invalid data gets submitted
    console.log(this.loginForm.value);
  }

  ngOnDestroy() {
    this.sub.unsubscribe(); // \u274c easy to forget; doesn't scale
  }
}
```

What breaks: No validators means the form is always valid \u2014 invalid data reaches the API. Manual `ngOnDestroy` unsubscribe is brittle and often forgotten. After 10 navigations to this component, 10 active subscriptions are firing simultaneously.

\u2705 After \u2014 FormBuilder, validators, takeUntilDestroyed, submit guard

```
export class NewFormComponent {
  private fb = inject(FormBuilder);

  loginForm = this.fb.group({
    email:    ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(8)]],
  });

  constructor() {
    // \u2705 In constructor \u2014 injection context active, auto-cleanup guaranteed
    this.loginForm.valueChanges
```

```
      .pipe(debounceTime(300), takeUntilDestroyed())
      .subscribe(val => console.log(val));
  }

  onSubmit() {
    if (this.loginForm.invalid) {
      this.loginForm.markAllAsTouched(); // \u2705 reveal hidden errors
      return;
    }
    console.log(this.loginForm.value);
  }
}
```

Why this matters: `takeUntilDestroyed()` eliminates `ngOnDestroy` boilerplate entirely. Validators make the form's validity state meaningful. `markAllAsTouched()` surfaces errors for users who submit without interacting with fields. `FormBuilder` reduces noise and makes the structure scannable at a glance.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Forgetting to import `ReactiveFormsModule`

What it is: The component renders an input with `formControlName` but Angular throws `Can't bind to 'formControlName' since it isn't a known property`.

Why it happens: The `FormGroup` is set up in TypeScript but the template directives (`formControlName`, `formGroup`, `formArrayName`) live in `ReactiveFormsModule` \u2014 which must be explicitly imported in standalone components.

How to avoid: Always add `ReactiveFormsModule` to the `imports` array of the standalone component.

Angular consequence: The form silently doesn't bind \u2014 inputs are uncontrolled, values are never captured, and the error only appears in the console.

Mistake 2 \u2014 Calling `takeUntilDestroyed()` outside the injection context

What it is: Placing `takeUntilDestroyed()` inside `ngOnInit()` instead of the constructor or a field initializer.

Why it happens: `ngOnInit` feels like "the right place to set things up." Developers move subscription code there without realising `takeUntilDestroyed()` requires the DI injection context, which only exists during construction.

How to avoid: Subscribe to `valueChanges` / `statusChanges` in the constructor. If you need `DestroyRef` in a method, inject it explicitly: `private destroyRef = inject(DestroyRef)` then `takeUntilDestroyed(this.destroyRef)`.

Angular consequence: Runtime error \u2014 `takeUntilDestroyed() can only be used within an injection context`. If the developer falls back to manual `ngOnDestroy` unsubscribe and

forgets it, the subscription leaks.

Mistake 3 \u2014 Using `setValue` when `patchValue` is needed

What it is: Calling `form.setValue({ email: 'x@x.com' })` on a `FormGroup` that has more fields than just `email`.

Why it happens: Both methods look similar. `setValue` feels like "set a value."

How to avoid: `setValue` requires every control in the group to be provided \u2014 missing any throws an error. `patchValue` updates only the controls you provide \u2014 safe for partial updates from API responses.

Angular consequence: `setValue` throws at runtime \u2014 `Must supply a value for form control with name: 'password'` \u2014 visible in production when populating a form from a partial API payload.

Mistake 4 \u2014 Accessing `form.value` on submit without guarding

What it is: `onSubmit() { this.api.register(this.form.value); }` \u2014 no validity check, no `markAllAsTouched()`.

Why it happens: The submit button has `[disabled]="form.invalid"` \u2014 looks safe. But `[disabled]` can be bypassed via browser DevTools, and validation errors only show on touched fields \u2014 a user who submits via keyboard sees no feedback.

How to avoid: Always guard with `if (this.form.invalid) { this.form.markAllAsTouched(); return; }`.

Angular consequence: Invalid data reaches the API. The backend rejects it, the UI has no visible feedback. Users have no idea what went wrong.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Use `inject(FormBuilder)` to reduce boilerplate	Manually `new FormControl()` / `new FormGroup()` in every component
Subscribe to `valueChanges` in the constructor with `takeUntilDestroyed()`	Subscribe in `ngOnInit` without cleanup \u2014 guaranteed memory leak
Use `patchValue` when pre-filling from API data (partial updates)	Use `setValue` for partial updates \u2014 it throws if any control is missing
Call `markAllAsTouched()` in `onSubmit()` before returning on invalid	Trust `[disabled]` alone \u2014 it can be bypassed via browser DevTools

\u2705 DO	\u274c DON'T
Use getter accessors (`get email() { return this.form.get('email')! }`) for cleaner templates	Repeat `profileForm.get('email')` inline in the template multiple times
Use `form.getRawValue()` when you need disabled control values	Use `form.value` \u2014 it silently omits disabled controls
Apply cross-field validators at the `FormGroup` level	Try to access sibling controls from within a `FormControl`-level validator

---

Section 7 \u2014 Interview Prep

Question 1

"What is the difference between `setValue` and `patchValue` in Reactive Forms, and when would you use each?"

Strong answer covers:

- `setValue` requires all controls to be supplied \u2014 throws if any are missing
- `patchValue` accepts a partial object \u2014 silently ignores missing keys
- Use `setValue` when replacing the entire form (e.g. loading a saved record where all fields are known)
- Use `patchValue` when pre-filling from a partial API response or updating a subset of fields
- Both work on `FormControl`, `FormGroup`, and `FormArray`
- `form.reset(defaultValues)` is a third option \u2014 clears dirty/touched state as well

Weak answer misses: That `setValue` throws on missing fields \u2014 candidates often say it "ignores them" which is wrong.

Level: Junior

Question 2

"How do you handle dynamic form fields in Angular \u2014 fields that are added or removed at runtime based on user actions?"

Strong answer covers:

- `FormArray` is the correct primitive for runtime-dynamic lists
- `push(control)` adds, `removeAt(index)` removes
- `formArrayName` directive on the container + `[formControlName]="$index"` for simple controls
- `[formGroupName]="$index"` for nested `FormGroup` items inside the array
- `FormBuilder.array([])` for cleaner initialization

- Parent form validity is automatically affected when array contents change \u2014 pushing an invalid control immediately makes the parent invalid

Weak answer misses: The template binding requirement (`formArrayName` + `[formGroupName]` for nested groups) \u2014 many candidates know the TypeScript side but can't wire up the template correctly.

Level: Mid

Question 3

"How would you implement a cross-field validator \u2014 for example, validating that a `confirmPassword` field matches the `password` field?"

Strong answer covers:

- Cross-field validators are applied at the `FormGroup` level, not the `FormControl` level
- The validator receives the `AbstractControl` (the group) and calls `.get('field')` to access siblings
- Return `null` for valid, return an error object (`{ passwordMismatch: true }`) for invalid
- The error lives on the group: `form.errors?.['passwordMismatch']` \u2014 not on `confirmPassword.errors`
- For reusability, write as a standalone function and pass to the `validators` option of `fb.group()`

Weak answer misses: That the error is set on the group, not on the `confirmPassword` control \u2014 a critical distinction for template error display.

Level: Mid\u2013Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Reactive Forms define your form as a TypeScript class tree \u2014 `FormControl` for individual fields, `FormGroup` for related sets of fields, `FormArray` for dynamic lists. The template binds to this model; all state, validation, and reactivity live in the class. Validity propagates bottom-up \u2014 one invalid child makes the entire parent invalid.

Syntax at a glance:

Method / Property	What it does
`setValue(obj)`	Sets entire form value \u2014 all fields required or throws
`patchValue(obj)`	Sets partial form value \u2014 missing fields untouched

Method / Property	What it does
`reset()`	Clears value, marks pristine and untouched
`getRawValue()`	Gets value including disabled controls (`value` excludes them)
`markAllAsTouched()`	Forces all error messages to show at once (use in `onSubmit`)
`disable()` / `enable()`	Disables/enables the control and removes it from `.value`
`addControl()` / `removeControl()`	Dynamically mutate a `FormGroup`
`push()` / `removeAt()`	Dynamically mutate a `FormArray`

Choosing between `setValue` and `patchValue`:

Situation	Use
Pre-filling from a full API response	`setValue`
Pre-filling from a partial API response	`patchValue`
Updating one field programmatically	`patchValue`
Resetting the entire form to known defaults	`reset(defaultValues)`
Updating a single `FormControl`	`control.setValue(value)` directly

When to use / when not to use:

Use Reactive Forms for any non-trivial form \u2014 login, registration, multi-step wizards, dynamic field arrays, forms driven by API data. Use Template-Driven Forms only for simple single-field scenarios where testability and programmatic control are not required.

The one rule that matters most:

Never subscribe to `valueChanges` without `takeUntilDestroyed()` \u2014 every unmanaged form subscription is a memory leak that compounds with every component navigation in production.

Next topic in order: Form Validators (Phase 5, Topic 40 in your plan).

FORM VALIDATORS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 The built-in validators are straightforward, but custom synchronous validators require understanding `AbstractControl`, and async validators add RxJS Observables into the mix \u2014 making this a multi-layered topic that builds directly on Reactive Forms.

Angular Relevance: Critical \u2014 Every real-world form needs validation beyond "is this field filled in?" Custom and async validators are what separates a production-grade Angular form from a toy example \u2014 password rules, username availability checks, cross-field comparisons all live here.

Section 1 \u2014 Explanation

The Bouncer Analogy

Imagine a nightclub with three types of bouncers at the door.

Bouncer Type 1 \u2014 The Rulebook Bouncer (Built-in Validators): Has a printed rulebook. Checks instantly: "Are you over 18? Is your ID not expired?" Fast, synchronous, no waiting. Angular ships with these: `required`, `email`, `minLength`, `maxLength`, `min`, `max`, `pattern`.

Bouncer Type 2 \u2014 The Custom Rulebook Bouncer (Custom Sync Validators): You wrote the rules yourself. "Must be wearing a collared shirt AND shoes." Still checks instantly \u2014 no waiting \u2014 but the rules are yours, not from the printed book.

Bouncer Type 3 \u2014 The Radio Bouncer (Async Validators): Has to radio the VIP list database before letting someone in. Takes a moment. The door says "checking\u2026" (`pending` state) before the answer comes back. This is an async validator \u2014 it makes an HTTP call and returns an Observable or Promise.

All three bouncers return the same answer: `null` (you're in \u2014 valid) or **an error object** (you're not \u2014 here's why).

Technical Explanation

Every validator \u2014 built-in, custom sync, or custom async \u2014 follows one contract:

```
ValidatorFn:      (control: AbstractControl) => ValidationErrors | null
AsyncValidatorFn: (control: AbstractControl) => Observable<ValidationErrors | null>
                                              | Promise<ValidationErrors | null>
```

`ValidationErrors` is just `{ [key: string]: any }` \u2014 an object where the key is the error name and the value is any metadata.

Validator Type	Returns	When it runs	`pending` state?	
Built-in (sync)	`ValidationErrors \	null`	On every value change	No
Custom sync	`ValidationErrors \	null`	On every value change	No
Custom async	`Observable<...> \	Promise<...>`	After all sync validators pass	Yes \u2014 until resolved

Critical rule: Angular only runs async validators if all synchronous validators pass first. This prevents unnecessary HTTP calls when the field is obviously invalid.

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular validators are standalone functions \u2014 no classes, no decorators. They are passed directly into `fb.group()` or `new FormControl()`. Async validators use `inject()` to access services when defined as factory functions.

Migration table:

Legacy Pattern	Modern Replacement
Validator class with `implements Validator`	Plain `ValidatorFn` function
`provide: NG_VALIDATORS` registration	Pass directly to `fb.group()` or `FormControl`
Constructor injection in validator class	Factory function pattern: `function myValidator(service: MyService): ValidatorFn`
`*ngIf` for error display	`@if (control.invalid &amp;&amp; control.touched)`
`updateOn: 'change'` (default)	Consider `updateOn: 'blur'` for async validators to reduce API calls

---

Section 3 \u2014 Code Example

Plain TS \u2014 validator contract in isolation

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

// A validator is just a function \u2014 nothing more
function noSpacesValidator(control: AbstractControl): ValidationErrors | null {
  const hasSpaces = (control.value as string)?.includes(' ');
  // null = valid. Error object = invalid. Key is the error name.
  return hasSpaces ? { noSpaces: { actualValue: control.value } } : null;
}

const usernameControl = new FormControl('hello world', [noSpacesValidator]);
```

```
console.log(usernameControl.errors); // { noSpaces: { actualValue: 'hello world' } }
console.log(usernameControl.valid);  // false

const cleanControl = new FormControl('helloworld', [noSpacesValidator]);
console.log(cleanControl.valid);     // true
```

`validators/custom-validators.ts` **\u2014 all three types**

```
import { AbstractControl, ValidationErrors, ValidatorFn, AsyncValidatorFn } from '@angular/forms';
import { Observable, of, timer } from 'rxjs';
import { switchMap, map, catchError } from 'rxjs/operators';
import { UserService } from '../services/user.service';

// \u2500\u2500 CUSTOM SYNC VALIDATOR \u2014 parameterised factory \u2500\u2500\u2500\u2500\u2500\u2500\u2500

// Returns a ValidatorFn \u2014 the factory pattern for reusable validators with config
export function forbiddenNameValidator(forbiddenNames: string[]): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const isForbidden = forbiddenNames.includes(control.value?.toLowerCase());
    return isForbidden
      ? { forbiddenName: { value: control.value } }  // error object
      : null;                                          // valid
  };
}

// \u2500\u2500 CROSS-FIELD SYNC VALIDATOR \u2014 applied to FormGroup \u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500

// Receives the group as AbstractControl \u2014 accesses siblings via .get()
export function passwordMatchValidator(group: AbstractControl): ValidationErrors | null {
  const password        = group.get('password')?.value;
  const confirmPassword = group.get('confirmPassword')?.value;
  // Error lands on the GROUP, not on any individual control
  return password === confirmPassword ? null : { passwordMismatch: true };
}

// \u2500\u2500 ASYNC VALIDATOR \u2014 username availability via HTTP \u2500\u2500\u2500\u2500\u2500\u2500\u2500\u2500

export function usernameAvailabilityValidator(
  userService: UserService
): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    // Guard: let Validators.required handle empty \u2014 don't hit API on empty value
    if (!control.value) return of(null);

    return timer(400).pipe(                            // debounce 400ms
      switchMap(() =>
        userService.checkUsernameAvailable(control.value)
      ),
      map(isAvailable => isAvailable ? null : { usernameTaken: true }),
      // MANDATORY: if the HTTP call fails, resolve as valid
      // Without this: control stays in PENDING state forever
      catchError(() => of(null))
    );
  };
}
```

`register.component.ts` **\u2014 all three types wired together**

```
import { Component, inject } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, Validators, AbstractControl } from '@angular/forms';
import {
  forbiddenNameValidator,
  passwordMatchValidator,
  usernameAvailabilityValidator
} from './validators/custom-validators';
import { UserService } from './services/user.service';

@Component({
  selector: 'app-register',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="registerForm" (ngSubmit)="onSubmit()">

      <input formControlName="username" placeholder="Username" />

      <!-- Show pending state while async validator is running -->
      @if (username.pending) {
        <small>Checking availability\u2026</small>
      }
      @if (username.invalid && username.touched && !username.pending) {
        <small>
          @if (username.errors?.['required'])      { Username is required }
          @if (username.errors?.['minlength'])     { Must be at least 3 characters }
          @if (username.errors?.['forbiddenName']) { That username is not allowed }
          @if (username.errors?.['usernameTaken']) { Username is already taken }
        </small>
      }

      <input formControlName="password" type="password" placeholder="Password" />
      <input formControlName="confirmPassword" type="password" placeholder="Confirm" />

      <!-- Cross-field error lives on the GROUP \u2014 not on confirmPassword.errors -->
      @if (registerForm.errors?.['passwordMismatch'] &&
           registerForm.get('confirmPassword')?.touched) {
        <small>Passwords do not match</small>
      }

      <!-- Disable while form is invalid OR while async validation is in progress -->
      <button type="submit" [disabled]="registerForm.invalid || registerForm.pending">
        Register
      </button>
    </form>
  `,
})
export class RegisterComponent {
  private fb          = inject(FormBuilder);
  private userService = inject(UserService);

  registerForm = this.fb.group(
    {
      username: [
        '',
        // 2nd argument: sync validators array
        [Validators.required, Validators.minLength(3), forbiddenNameValidator(['admin', 'root'])],
        // 3rd argument: async validators array \u2014 only run after sync validators pass
        [usernameAvailabilityValidator(this.userService)]
```

```
      ],
      password:        ['', [Validators.required, Validators.minLength(8)]],
      confirmPassword: ['', Validators.required],
    },
    // Group-level validator \u2014 receives the whole group, not a single control
    { validators: passwordMatchValidator }
  );

  get username() { return this.registerForm.get('username')!; }

  onSubmit(): void {
    if (this.registerForm.invalid) {
      this.registerForm.markAllAsTouched(); // reveal all hidden errors
      return;
    }
    console.log(this.registerForm.value);
  }
}
```

Password strength validator with multi-rule feedback

```
// validators/password-strength.validator.ts
export function passwordStrengthValidator(control: AbstractControl): ValidationErrors | null {
  const value: string = control.value || '';
  if (!value) return null; // let required handle empty

  const errors: ValidationErrors = {};

  if (value.length < 8)          errors['minLength']       = true;
  if (!/[A-Z]/.test(value))      errors['requiresUpper']   = true;
  if (!/[0-9]/.test(value))      errors['requiresNumber']  = true;
  if (!/[!@#$%^&*]/.test(value)) errors['requiresSpecial'] = true;

  return Object.keys(errors).length > 0 ? errors : null;
}

<!-- Template: show each unmet rule as a live checklist -->
<input formControlName="password" type="password" />
@if (password.errors && password.dirty) {
  <ul class="password-rules">
    <li [class.met]="!password.errors?.['minLength']">      At least 8 characters </li>
    <li [class.met]="!password.errors?.['requiresUpper']">  One uppercase letter   </li>
    <li [class.met]="!password.errors?.['requiresNumber']"> One number             </li>
    <li [class.met]="!password.errors?.['requiresSpecial']">One special character  </li>
  </ul>
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 imperative validation in `onSubmit`

```
export class OldRegisterComponent {
  registerForm = this.fb.group({
    username: [''],  // \u274c no validators \u2014 always valid
    email:    ['']
  });
```

```
  onSubmit() {
    const { username, email } = this.registerForm.value;

    // \u274c Validation done imperatively at submit time \u2014 not reactive
    if (!username || username.length < 3) {
      alert('Username must be at least 3 characters');
      return;
    }
    if (!email || !email.includes('@')) {
      alert('Invalid email');
      return;
    }
  }
}
```

What breaks: No live feedback as the user types \u2014 errors only appear on submit. `[disabled]` binding can't work because the form is always valid. Logic is locked inside `onSubmit` \u2014 impossible to unit test the validation rules in isolation. Scaling to 10+ fields is unmanageable.

\u2705 After \u2014 validators declared on the form model

```
export class NewRegisterComponent {
  registerForm = this.fb.group({
    username: ['', [Validators.required, Validators.minLength(3)]],
    email:    ['', [Validators.required, Validators.email]],
  });

  onSubmit() {
    if (this.registerForm.invalid) {
      this.registerForm.markAllAsTouched(); // \u2705 reveal all hidden errors at once
      return;
    }
    // submit \u2014 form is guaranteed valid here
  }
}
```

Why this matters: The form's `valid`/`invalid` state is live \u2014 `[disabled]="form.invalid"` works correctly. Template shows inline errors per field as the user types. Validator functions are standalone \u2014 unit-testable without instantiating the component. `markAllAsTouched()` surfaces errors for users who skip fields and click submit directly.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Returning `false` or `true` instead of `null` or an error object

What it is: `if (control.value === 'bad') return false;` inside a validator.

Why it happens: Coming from native HTML validation or other frameworks where `true`/`false` signals valid/invalid feels natural.

How to avoid: The contract is strict \u2014 return `null` for valid, return `{ errorKey: true }` for invalid. Angular only checks for `null` vs non-null.

Angular consequence: The validator silently does nothing. The field is always treated as valid. No error in the console \u2014 just broken validation with no explanation.

Mistake 2 \u2014 Not debouncing async validators

What it is: Async validator that directly calls `service.check(control.value)` without wrapping in `timer(400).pipe(switchMap(...))`.

Why it happens: Developers implement the happy path first without considering the validator runs on every single keystroke.

How to avoid: Always wrap with `timer(400).pipe(switchMap(() => service.check(control.value)))`. The timer debounces; `switchMap` cancels the previous in-flight request if a new keystroke arrives before 400ms.

Angular consequence: Typing "johndoe" fires 7 HTTP requests. Responses arrive out of order \u2014 an earlier response can overwrite a later one, showing the wrong validity state. API gets hammered.

Mistake 3 \u2014 Forgetting `catchError` in async validators

What it is: Async validator with no `catchError` \u2014 if the HTTP call fails, the Observable errors out.

Why it happens: Error handling feels like an edge case. The happy path works fine in development.

How to avoid: Always pipe `catchError(() => of(null))` at the end of every async validator \u2014 no exceptions.

Angular consequence: If the API returns a 500 or the network drops, the Observable errors and Angular's internal subscription dies \u2014 the control stays in `PENDING` state forever. The submit button stays permanently disabled. Users are stuck with no way to proceed.

Mistake 4 \u2014 Looking for cross-field errors on the child control

What it is: `form.get('confirmPassword')?.errors?.['passwordMismatch']` \u2014 looking for the group-level error on the individual control.

Why it happens: It feels natural to check the confirm password field for the mismatch error.

How to avoid: Group-level validators set errors on the **group**, not on child controls. Always check `form.errors?.['passwordMismatch']`.

Angular consequence: The error message never appears \u2014 `confirmPassword.errors` is `null` even when passwords don't match. The form appears to have no validation for the most important check.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Return `null` for valid, `{ errorKey: true }` for invalid \u2014 always	Return `true`, `false`, or `undefined` from a validator
Put reusable validators in a dedicated `validators/` file	Inline one-off validator logic directly in the component class
Add `catchError(() => of(null))` to every async validator	Let async validators throw \u2014 they permanently lock the form in `pending`
Debounce async validators with `timer(400).pipe(switchMap(...))`	Fire an HTTP request on every single keystroke
Apply cross-field validators at the `FormGroup` level	Apply cross-field validators to an individual `FormControl` \u2014 you can't access siblings from there
Check `form.errors?.['key']` for group-level errors in the template	Check `control.errors?.['key']` for a cross-field error \u2014 it won't be there
Guard for `null`/empty in custom validators \u2014 let `Validators.required` handle empty	Call methods on `control.value` without a null check \u2014 throws on initial render
Show `pending` state in the template and disable submit while async validation runs	Leave users with no indication the form is checking something

---

Section 7 \u2014 Interview Prep

Question 1

"What is the difference between a synchronous and an asynchronous validator in Angular, and when would you use each?"

Strong answer covers:

- Sync validators run immediately on every value change; async validators return an Observable/Promise and run only after all sync validators pass
- The `pending` status exists only during async validation \u2014 use it in templates to show "checking\u2026" feedback
- Use sync for format/length/pattern checks; use async for server-side uniqueness checks (email, username availability)
- Async validators should always be debounced with `timer(N).pipe(switchMap(...))` to prevent API hammering
- `catchError(() => of(null))` is mandatory in every async validator \u2014 without it the form can get stuck in `pending`

- Angular skips async validators when any sync validator fails \u2014 always add `Validators.required` before an async validator

Weak answer misses: That Angular intentionally skips async validators when sync validators fail \u2014 and why that matters for ordering validators correctly.

Level: Mid

Question 2

"How do you implement a cross-field validator \u2014 one that compares two fields in the same form group?"

Strong answer covers:

- Applied at the `FormGroup` level via the `validators` option of `fb.group()` \u2014 not on an individual `FormControl`
- The validator receives the group as `AbstractControl` \u2014 call `.get('fieldName')` to access child controls
- Returns `null` or an error object \u2014 same contract as all validators
- The error lands on the **group's** `.errors`, not on either child control
- Template must check `form.errors?.['errorKey']` not `field.errors?.['errorKey']`
- Gating the error display on `form.get('confirmPassword')?.touched` prevents showing the error before the user interacts

Weak answer misses: Where the error actually lands (the group, not the child) \u2014 the single most common implementation mistake.

Level: Mid

Question 3

"A user reports that after a network error, the form's submit button stays permanently disabled. What would you investigate and how would you fix it?"

Strong answer covers:

- Root cause: an async validator's Observable threw an error, Angular's internal subscription died, the control is stuck in `PENDING` state
- Diagnosis: check `control.status === 'PENDING'` in DevTools, check the Network tab for red (failed) API calls
- Fix: `catchError(() => of(null))` \u2014 ensures the Observable always resolves with a value, never throws
- Treating API errors as `of(null)` (valid) is the correct safe default \u2014 the server will re-validate on submit

- Team standard: `catchError` should be mandatory on every async validator \u2014 enforced at code review

Weak answer misses: The mechanism \u2014 understanding that the Observable completing vs erroring is what drives the status transition out of `PENDING`.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

A validator is a function that receives a `FormControl` or `FormGroup` and returns `null` for valid or `{ errorKey: true }` for invalid \u2014 built-in, custom sync, and async all follow this exact contract. Async validators return an Observable and put the control in `pending` state until resolved. Cross-field validators are applied to the `FormGroup` and their errors land on the group, not on any child control.

Validator placement cheat sheet:

Validator type	Where to attach	Where error appears
Single-field sync	2nd arg of `fb.group` field	`control.errors`
Single-field async	3rd arg of `fb.group` field	`control.errors`
Cross-field sync	`validators` option of `fb.group`	`formGroup.errors`
Cross-field async	`asyncValidators` option of `fb.group`	`formGroup.errors`

Sync vs async at a glance:

Sync Validator	Async Validator			
Return type	`ValidationErrors \	null`	`Observable<Validation Errors \	null>`
When it runs	Every value change	After all sync validators pass		
`pending` state	No	Yes \u2014 while Observable is resolving		
Placement in `fb.group`	2nd argument array	3rd argument array		
Typical use	Format, length, pattern	API calls (username taken, email exists)		

When to use / when not to use:

Use built-in validators for common cases (`required`, `email`, `minLength`). Use custom sync validators for business-specific rules that can be checked locally. Use async validators only for server-side checks \u2014 not for anything computable client-side. Never use async validators without debouncing and `catchError`.

The one rule that matters most:

Every async validator must end with `catchError(() => of(null))` \u2014 a thrown error permanently locks the control in `PENDING` state and disables the form with no way for the user to recover.

Next topic in order: Template-Driven Forms (Phase 5, Topic 41 in your plan).

TEMPLATE-DRIVEN FORMS

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Beginner\u2013Intermediate \u2014 The template syntax is approachable, but understanding why it works \u2014 and where it breaks down compared to Reactive Forms \u2014 requires a solid mental model of how Angular's directive system binds to an implicit form model behind the scenes.

Angular Relevance: High \u2014 Template-Driven Forms are the right tool for simple, low-logic forms. You'll encounter them in legacy codebases, third-party component libraries, and anywhere rapid prototyping matters. Understanding them also sharpens your mental model of Reactive Forms by contrast.

Section 1 \u2014 Explanation

The Verbal Instructions Analogy

Recall the Blueprint Analogy from Reactive Forms. Template-Driven Forms are the other approach \u2014 the Verbal Instructions approach.

Instead of drawing a blueprint in TypeScript before construction begins, you walk onto the site and tell Angular what to build by annotating the HTML directly: "This input is called username. Make it required. Minimum 3 characters." Angular listens, builds the form model for you behind the scenes, and keeps it in sync with the template.

Simple shed? Verbal instructions work great. You barely need a blueprint. Complex skyscraper with dynamic floors? You'll wish you had a blueprint. That's when Reactive Forms win.

Template-Driven Forms are Angular saying: "You describe the form in HTML \u2014 I'll create the model." Reactive Forms are you saying: "I'll create the model in TypeScript \u2014 you just connect the template to it."

Technical Explanation

When you add `FormsModule` and use `ngModel` on an input, Angular automatically:

1. Creates a `FormControl` for that input behind the scenes
2. Creates a `FormGroup` for the enclosing `<form>` element (via the `NgForm` directive)
3. Registers each `ngModel` control into that implicit group using the `name` attribute
4. Keeps the DOM value and the implicit model in sync (two-way binding)

You don't see the `FormControl` objects \u2014 Angular manages them. You interact with the model through template reference variables (`#myForm="ngForm"`) and the `ngModel` directive.

Template-Driven	Reactive	
Form model created by	Angular (implicit, in template)	You (explicit, in TypeScript)

Template-Driven	Reactive	
Primary module	`FormsModule`	`ReactiveFormsModule`
Two-way binding	`[(ngModel)]` \u2014 built in	Manual (`formControlName` + event)
Validation	HTML attributes + directive-based	Validator functions in TypeScript
Access in component	Via `@ViewChild` or template ref	Direct property access
Testability	Harder (requires DOM)	Easy (pure TypeScript)
Best for	Simple, low-logic forms	Complex, dynamic, testable forms

---

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Template-Driven Forms use standalone components with `FormsModule` in the `imports` array \u2014 no `NgModule` needed. Template control flow uses `@if` and `@for` instead of `*ngIf` and `*ngFor`. `inject()` replaces constructor injection for any services used in the component.

Migration table:

Legacy Pattern	Modern Replacement
`NgModule` with `FormsModule` in `imports`	`FormsModule` in standalone component `imports` array
`*ngIf="field.invalid && field.touched"`	`@if (field.invalid && field.touched)`
`*ngFor` over select options	`@for (option of options; track option.id)`
Constructor injection of services	`private service = inject(MyService)`
`@Input()` decorator	`input()` signal function (for component inputs, not form inputs)

Note: `[(ngModel)]` itself is not deprecated \u2014 it remains the correct API for Template-Driven Forms. What changes in modern Angular is the surrounding component and template infrastructure, not the form binding syntax.

Section 3 \u2014 Code Example

Plain concept \u2014 what `[(ngModel)]` actually desugars to

```
<!-- These two are identical: -->

<!-- Shorthand (banana-in-a-box) -->
<input [(ngModel)]="username" />
```

```
<!-- Longhand (what Angular actually executes) -->
<input [ngModel]="username" (ngModelChange)="username = $event" />

// Component class \u2014 just a plain property, no FormControl needed
export class MyComponent {
  username = ''; // Angular keeps this in sync with the input automatically
}
```

`contact-form.component.ts` \u2014 complete working example

```
import { Component, inject } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

interface ContactFormData {
  name:    string;
  email:   string;
  message: string;
  consent: boolean;
}

@Component({
  selector: 'app-contact-form',
  standalone: true,
  imports: [FormsModule], // FormsModule \u2014 NOT ReactiveFormsModule
  template: `
    <!--
      #contactForm="ngForm" creates a template reference to the NgForm directive
      Angular auto-attaches NgForm to every <form> element when FormsModule is imported
    -->
    <form #contactForm="ngForm" (ngSubmit)="onSubmit(contactForm)">

      <!--
        name="name" is REQUIRED \u2014 it registers the control in NgForm under this key
        #nameField="ngModel" \u2014 ref to the NgModel directive instance, not the DOM element
      -->
      <input
        name="name"
        [(ngModel)]="formData.name"
        required
        minlength="2"
        #nameField="ngModel"
      />
      @if (nameField.invalid && nameField.touched) {
        <small>
          @if (nameField.errors?.['required'])  { Name is required }
          @if (nameField.errors?.['minlength']) {
            Minimum {{ nameField.errors?.['minlength'].requiredLength }} characters
          }
        </small>
      }

      <input
        name="email"
        type="email"
        [(ngModel)]="formData.email"
        required
        email
        #emailField="ngModel"
```

```
      />
      @if (emailField.invalid && emailField.touched) {
        <small>
          @if (emailField.errors?.['required']) { Email is required }
          @if (emailField.errors?.['email'])    { Must be a valid email address }
        </small>
      }

      <textarea
        name="message"
        [(ngModel)]="formData.message"
        required
        minlength="10"
        #messageField="ngModel"
      ></textarea>
      @if (messageField.invalid && messageField.touched) {
        <small>Message must be at least 10 characters</small>
      }

      <label>
        <input
          name="consent"
          type="checkbox"
          [(ngModel)]="formData.consent"
          required
          #consentField="ngModel"
        />
        I agree to the terms
      </label>
      @if (consentField.invalid && consentField.touched) {
        <small>You must accept the terms</small>
      }

      <!-- contactForm.invalid uses the NgForm ref to check overall validity -->
      <button type="submit" [disabled]="contactForm.invalid">Send</button>
    </form>
  `,
})
export class ContactFormComponent {
  // Plain component properties \u2014 no FormControl, no FormGroup, no FormBuilder
  formData: ContactFormData = {
    name:    '',
    email:   '',
    message: '',
    consent: false,
  };

  onSubmit(form: NgForm): void {
    if (form.invalid) {
      form.control.markAllAsTouched(); // reveal all hidden errors
      return;
    }
    console.log('Submitted:', this.formData);
    form.reset(); // clears values AND resets dirty/touched/pristine
  }
}
```

Accessing `NgForm` from the component class via `@ViewChild`

```
import { Component, ViewChild, AfterViewInit, inject } from '@angular/core';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-edit-profile',
  standalone: true,
  imports: [FormsModule],
  template: `
    <form #profileForm="ngForm" (ngSubmit)="onSave(profileForm)">
      <input name="displayName" [(ngModel)]="profile.displayName"
             required minlength="2" #nameRef="ngModel" />
      @if (nameRef.invalid && nameRef.touched) {
        <small>Display name must be at least 2 characters</small>
      }

      <input name="bio" [(ngModel)]="profile.bio" maxlength="160" />

      <!-- pristine: disable Save if the user hasn't changed anything yet -->
      <button type="submit" [disabled]="profileForm.invalid || profileForm.pristine">
        Save Changes
      </button>
    </form>
  `,
})
export class EditProfileComponent implements AfterViewInit {
  // @ViewChild not available until ngAfterViewInit \u2014 never access in ngOnInit
  @ViewChild('profileForm') profileForm!: NgForm;

  private userService = inject(UserService);

  profile = { displayName: '', bio: '' };

  ngAfterViewInit(): void {
    this.userService.getProfile().subscribe(data => {
      // setTimeout avoids ExpressionChangedAfterItHasBeenChecked error
      // Pushes the update to the next change detection cycle
      setTimeout(() => {
        this.profileForm.setValue({
          displayName: data.displayName,
          bio:         data.bio ?? '',
        });
      });
    });
  }

  onSave(form: NgForm): void {
    if (form.invalid) return;
    this.userService.updateProfile(this.profile).subscribe(() => {
      form.markAsPristine(); // re-disable Save \u2014 no unsaved changes
    });
  }
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 missing `name` attribute, broken registration

```
<!-- WRONG: no name attribute \u2014 Angular can't register this control in NgForm -->
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <input [(ngModel)]="email" required />
  <button type="submit">Submit</button>
</form>
```

What breaks: Angular throws: `If ngModel is used within a form tag, either the name attribute must be set or the form control must be defined as 'standalone' in ngModelOptions`. The control is never registered in `NgForm` \u2014 `myForm.value` is `{}` and `myForm.valid` is always `true` because no controls are tracked.

\u2705 After \u2014 correct `name` attribute added

```
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <!-- name="email" registers this control under the key 'email' in NgForm -->
  <input name="email" [(ngModel)]="email" required #emailRef="ngModel" />
  @if (emailRef.invalid && emailRef.touched) {
    <small>Email is required</small>
  }
  <button type="submit" [disabled]="myForm.invalid">Submit</button>
</form>
```

Why this matters: The `name` attribute is the registration key. Without it, the control is invisible to the parent `NgForm` \u2014 the form's aggregate validity, value, and dirty state are all wrong. This is the most common beginner mistake with Template-Driven Forms.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Forgetting the `name` attribute on `ngModel` inputs

What it is: Using `[(ngModel)]` without `name="fieldName"` inside a `<form>` element.

Why it happens: `[(ngModel)]` works fine outside a form (standalone two-way binding), so developers assume it works the same inside one. Inside a form, `name` is the registration key \u2014 without it the control is invisible to `NgForm`.

How to avoid: Every `[(ngModel)]` inside a `<form>` must have a `name` attribute. Make it a personal code-review rule: no `ngModel` without `name`.

Angular consequence: Angular throws a runtime error. If somehow bypassed, `NgForm.value` is empty \u2014 submitted data is lost silently.

Mistake 2 \u2014 Trying to access `NgForm` before `ngAfterViewInit`

What it is: `@ViewChild('myForm') myForm!: NgForm` used in `ngOnInit()` \u2014 `myForm` is `undefined` at that point.

Why it happens: `ngOnInit` is habitually "where setup happens." Developers forget that `@ViewChild` queries aren't resolved until after the view is initialised.

How to avoid: Always use `@ViewChild`-based `NgForm` access in `ngAfterViewInit`. Wrap `setValue`/`patchValue` calls in `setTimeout()` to avoid `ExpressionChangedAfterItHasBeenChecked` errors.

Angular consequence: `TypeError: Cannot read properties of undefined` at runtime. The form never gets pre-populated.

Mistake 3 \u2014 Writing `#ref` without `="ngModel"`

What it is: `<input #nameField [(ngModel)]="name" />` \u2014 the template reference points to the DOM element, not the `NgModel` directive instance.

Why it happens: The `="ngModel"` part looks optional. It doesn't \u2014 without it `nameField` is an `HTMLInputElement`. `nameField.touched`, `nameField.errors` are `undefined`.

How to avoid: Always write `#fieldRef="ngModel"` (with the `="ngModel"` part) when you need access to the control's validity state.

Angular consequence: Validation UI never renders \u2014 `field.invalid &amp;&amp; field.touched` never evaluates correctly. Silent bug with no console error.

Mistake 4 \u2014 Mixing `[(ngModel)]` with Reactive Form directives

What it is: Using `ngModel` inside a `[formGroup]` container, or importing both `FormsModule` and `ReactiveFormsModule` and mixing their directives.

Why it happens: Both achieve input binding. Developers new to Angular don't yet distinguish which system each directive belongs to.

How to avoid: Pick one approach per form. Template-Driven \u2192 `FormsModule` + `ngModel` + `name`. Reactive \u2192 `ReactiveFormsModule` + `formControlName` + no `name` needed.

Angular consequence: Angular throws: `It looks like you're using ngModel on the same form field as formControlName`. Removed in Angular v7 \u2014 the app breaks at runtime.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Import `FormsModule` in standalone component `imports`	Import `ReactiveFormsModule` when using Template-Driven Forms \u2014 wrong module
Always add `name="fieldName"` to every `ngModel` input inside a `<form>`	Use `ngModel` inside a form without a `name` attribute
Use `#field="ngModel"` (with `="ngModel"`) for per-field error display	Write `#field` without `="ngModel"` \u2014 it gives you the DOM element, not the directive

\u2705 DO	\u274c DON'T
Pass the `NgForm` reference into `onSubmit(form: NgForm)` via the template	Access the `NgForm` in the component class without `@ViewChild`
Use `form.reset()` after successful submit \u2014 clears values AND marks pristine	Manually clear `formData` properties \u2014 leaves dirty/touched state stale
Access `@ViewChild NgForm` in `ngAfterViewInit` with `setTimeout()` for pre-population	Access `@ViewChild NgForm` in `ngOnInit` \u2014 it's `undefined` there
Switch to Reactive Forms when logic is complex, dynamic, or needs unit tests	Use Template-Driven Forms for dynamic fields, cross-field validation, or async checks

---

Section 7 \u2014 Interview Prep

Question 1

"What is the difference between Template-Driven and Reactive Forms in Angular? When would you choose one over the other?"

Strong answer covers:

- Template-Driven: implicit model, Angular creates `FormControl`/`FormGroup` from the template; Reactive: explicit model, developer creates the tree in TypeScript
- Template-Driven uses `FormsModule` + `ngModel`; Reactive uses `ReactiveFormsModule` + `formControlName`
- Reactive Forms are easier to unit test (no DOM required), support dynamic fields naturally, and scale better
- Template-Driven Forms are faster to write for simple, static forms with no logic
- Both use the same underlying `AbstractControl` model \u2014 only the authoring style differs
- Clear tiebreaker: if the form will ever need unit tests, use Reactive

Weak answer misses: That both systems use the same underlying `FormControl`/`FormGroup` classes \u2014 candidates often imply they are fundamentally different systems rather than different APIs over the same model.

Level: Junior\u2013Mid

Question 2

"In a Template-Driven Form, how do you access a specific field's validation state in the template, and what is the common mistake developers make?"

Strong answer covers:

- Template reference variable with `="ngModel"`: `#emailField="ngModel"`
- Without `="ngModel"`, the ref points to the DOM element, not the `NgModel` directive

- Access errors via `emailField.errors?.['required']`, `emailField.touched`, `emailField.invalid`
- The `name` attribute must be present for the control to be registered in `NgForm`
- Common mistake: writing `#emailField` without `="ngModel"` \u2014 ref is `HTMLInputElement`, `.touched` is `undefined`, validation UI never shows

Weak answer misses: The distinction between `#ref` (DOM element) and `#ref="ngModel"` (NgModel directive instance) \u2014 the defining gotcha of Template-Driven Form debugging.

Level: Junior

Question 3

"How would you pre-populate a Template-Driven Form with data loaded from an API, and what pitfalls would you watch out for?"

Strong answer covers:

- Use `@ViewChild('formRef') form!: NgForm` to get class-level access to the form
- `NgForm` is only available after `ngAfterViewInit` \u2014 not `ngOnInit`
- Wrap `setValue`/`patchValue` in `setTimeout()` to avoid `ExpressionChangedAfterItHasBeenChecked`
- Subscribe to the API call in `ngAfterViewInit` and apply the response to the form inside the `setTimeout`
- After saving, call `form.markAsPristine()` so the "unsaved changes" dirty state resets correctly

Weak answer misses: The `setTimeout` requirement and the reason for it \u2014 most candidates know to use `ngAfterViewInit` but don't know about the timing issue with `ExpressionChangedAfterItHasBeenChecked`.

Level: Mid

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Template-Driven Forms let Angular build the form model implicitly from your HTML using `ngModel` and `NgForm` directives. You annotate the template with `name`, validation attributes, and `[(ngModel)]` \u2014 Angular constructs the `FormControl`/`FormGroup` tree behind the scenes. The trade-off is less control and less testability, but far less code for simple cases.

Syntax at a glance:

Task	Template-Driven syntax
Two-way bind an input	`[(ngModel)]="property"`

Task	Template-Driven syntax
Register control in `NgForm`	`name="fieldName"` (required)
Access form state in template	`#formRef="ngForm"`
Access control state in template	`#fieldRef="ngModel"`
Check if field is invalid + touched	`fieldRef.invalid &amp;&amp; fieldRef.touched`
Access specific error	`fieldRef.errors?.['required']`
Submit handler	`(ngSubmit)="onSubmit(formRef)"`
Reset form	`formRef.reset()`
Access `NgForm` in class	`@ViewChild('formRef') form!: NgForm` \u2014 in `ngAfterViewInit` only

When to use / when not to use:

Signal	Use Template-Driven	Use Reactive
Form complexity	Simple, static fields	Dynamic, conditional fields
Validation needs	Built-in HTML attributes	Custom, cross-field, async
Testing requirement	Manual / e2e only	Unit testable
Code volume preference	Minimal TypeScript	More TS, cleaner at scale

The one rule that matters most:

Template-Driven Forms do not scale \u2014 use them only for simple, static, low-logic forms. Switch to Reactive Forms the moment you need dynamic fields, async validation, cross-field rules, or unit tests.

Next topic in order: Error Handling in Angular (Phase 5, Topic 42 in your plan).

ERROR HANDLING IN ANGULAR

Section 0 \u2014 Difficulty & Priority Rating

Difficulty: Intermediate \u2014 Individual pieces (try/catch, catchError, form validators) are familiar from prior topics, but knowing which error handling layer to use, when to let errors propagate, and how to prevent silent failures requires architectural thinking that only comes from seeing the full picture.

Angular Relevance: Critical \u2014 Every production Angular app deals with HTTP failures, form validation errors, and unexpected runtime exceptions. Getting this wrong means users see blank screens, frozen spinners, or no feedback at all. Getting it right means graceful degradation and a recoverable experience.

Section 1 \u2014 Explanation

The Building Fire Safety Analogy

Imagine a multi-storey office building. Errors in an Angular app are like fires \u2014 they can start anywhere, and you need different systems at different levels to contain them.

Sprinklers in each room (Component-level error handling): The first line of defence. If a small fire starts in one room, the sprinkler in that room handles it \u2014 the rest of the building is unaffected. This is your `error` callback in `.subscribe()`, your `@if (error)` in the template, your try/catch in a specific function.

Floor wardens (Service-level error handling): If the room sprinkler isn't enough, the floor warden steps in. `catchError` in your service intercepts HTTP errors before they reach the component \u2014 the service translates the technical failure into a domain-level message.

Building fire alarm system (Interceptor-level error handling): When an error needs a building-wide response \u2014 a 401 redirecting everyone to login, a 503 showing a global banner \u2014 the interceptor handles it. One place, every request.

The master emergency protocol (Global `ErrorHandler`): If a fire escapes all local systems and threatens to bring down the building, the master emergency protocol activates. Angular's `ErrorHandler` is the last catch \u2014 unhandled exceptions that would otherwise crash the app land here.

Four layers. Each has a job. None should try to do all four jobs alone.

Technical Explanation

Angular error handling operates across four distinct layers:

Layer	Tool	Scope	Use Case
Component	`error` callback, `@if (error)`	One component	Show retry button, error message
Service	`catchError` + `throwError`	One API domain	Translate HTTP codes to messages
Interceptor	`catchError` in interceptor	All HTTP requests	401 redirect, global toast
Global	`ErrorHandler` class	Entire app	Log uncaught exceptions, error page

The critical design principle: **errors should be handled at the lowest layer that has enough context to handle them meaningfully.** A 404 on a user detail page should probably be handled at the component level (show "User not found"). A 401 on any request should be handled at the interceptor level (redirect to login). An uncaught `TypeError` should be caught by the global handler.

Section 2 \u2014 Modern Angular Pattern (v17+)

Modern Angular error handling uses `inject()` inside functional interceptors, signal-based error state in components, and standalone `ErrorHandler` registration in `app.config.ts`. No class-based interceptors, no constructor injection.

Migration table:

Legacy Pattern	Modern Replacement		
Class-based interceptor with `implements HttpInterceptor`	Functional interceptor with `HttpInterceptorFn`		
`HTTP_INTERCEPTORS` token	`provideHttpClient(withInterceptors([...]))`		
`error: string \`	null = null` plain property	`error = signal<string \`	null>(null)`
Constructor injection in `ErrorHandler` subclass	`inject()` inside methods where needed		
`*ngIf="error"`	`@if (error())`		

Section 3 \u2014 Code Example

Layer 1 \u2014 Component-level error state with signals

```
// user-list.component.ts
```

```
import { Component, inject, signal } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { UserService, User } from './services/user.service';

@Component({
  selector: 'app-user-list',
  standalone: true,
  template: `
    @if (loading()) {
      <p>Loading\u2026</p>
    } @else if (error()) {
      <div class="error-banner">
        <p>{{ error() }}</p>
        <!-- Always give the user a way to recover -->
        <button (click)="loadUsers()">Retry</button>
      </div>
    } @else {
      @for (user of users(); track user.id) {
        <div>{{ user.name }}</div>
      }
    }
  `,
})
export class UserListComponent {
  private userService = inject(UserService);
  private destroyRef  = inject(DestroyRef);

  users   = signal<User[]>([]);
  loading = signal(false);
  error   = signal<string | null>(null);

  constructor() { this.loadUsers(); }

  loadUsers(): void {
    this.loading.set(true);
    this.error.set(null);    // clear previous error before retrying

    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next:  users => { this.users.set(users);  this.loading.set(false); },
        error: err   => { this.error.set(err.message); this.loading.set(false); },
      });
  }
}
```

Layer 2 \u2014 Service-level error translation with `catchError`

```
// user.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpErrorResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, map } from 'rxjs/operators';

export interface User { id: number; name: string; email: string; }

@Injectable({ providedIn: 'root' })
export class UserService {
  private http    = inject(HttpClient);
```

```
  private baseUrl = 'https://api.example.com';

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(`${this.baseUrl}/users`).pipe(
      catchError(this.handleError)
    );
  }

  getUserById(id: number): Observable<User> {
    return this.http.get<User>(`${this.baseUrl}/users/${id}`).pipe(
      catchError(this.handleError)
    );
  }

  // Centralised error translator \u2014 components never see raw HttpErrorResponse
  private handleError(error: HttpErrorResponse): Observable<never> {
    let message = 'An unexpected error occurred';

    if (error.status === 0) {
      // status 0 = network error \u2014 request never reached the server
      message = 'Network error \u2014 please check your connection';
    } else if (error.status === 404) {
      message = 'Resource not found';
    } else if (error.status === 401) {
      message = 'Unauthorised \u2014 please log in again';
    } else if (error.status === 403) {
      message = 'You do not have permission to perform this action';
    } else if (error.status >= 500) {
      message = 'Server error \u2014 please try again later';
    }

    console.error('[UserService] HTTP Error:', error.status, error.url);
    // throwError re-throws as an Observable error for the component's error callback
    return throwError(() => new Error(message));
  }
}
```

Layer 3 \u2014 Interceptor-level global error handling

```
// interceptors/error.interceptor.ts
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject }       from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { Router }       from '@angular/router';
import { ToastService } from '../services/toast.service';

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router       = inject(Router);
  const toastService = inject(ToastService);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      // 401 \u2014 token expired or invalid: redirect to login
      // Guard: don't redirect if already on login page or it's a login request
      const isLoginRequest = req.url.includes('/auth/login');
      const isLoginPage    = router.url.includes('/login');

      if (error.status === 401 && !isLoginRequest && !isLoginPage) {
        router.navigate(['/login'], {
```

```
          queryParams: { returnUrl: router.url }
        });
      }

      // 503 \u2014 service unavailable: show global toast
      if (error.status === 503) {
        toastService.show(
          'Service temporarily unavailable \u2014 please try again shortly',
          'warning'
        );
      }

      // Always re-throw \u2014 service-level catchError and component error callbacks
      // must still be able to handle the error. Never swallow it here.
      return throwError(() => error);
    })
  );
};
```

Layer 4 \u2014 Global `ErrorHandler` for uncaught exceptions

```
// error-handling/global-error-handler.ts
import { ErrorHandler, Injectable, inject } from '@angular/core';
import { Router } from '@angular/router';

@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  // inject() works in the handleError method via runInInjectionContext
  // For Router access, use a service that wraps it instead of injecting directly here
  private router = inject(Router);

  handleError(error: unknown): void {
    // Log to external monitoring service (Sentry, Datadog, etc.)
    console.error('[GlobalErrorHandler] Uncaught error:', error);

    // Optionally navigate to an error page for severe failures
    // Be conservative \u2014 only redirect for truly unrecoverable errors
    if (error instanceof Error && error.message.includes('ChunkLoadError')) {
      // Lazy chunk failed to load \u2014 app update during user session
      // Reload so user gets the latest version
      window.location.reload();
    }
  }
}

// app.config.ts \u2014 registering the global handler
import { ApplicationConfig, ErrorHandler } from '@angular/core';
import { provideRouter }      from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { GlobalErrorHandler } from './error-handling/global-error-handler';
import { errorInterceptor }   from './interceptors/error.interceptor';
import { APP_ROUTES }         from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(APP_ROUTES),
    provideHttpClient(
      withInterceptors([errorInterceptor])
```

```
    ),
    // Replace Angular's default ErrorHandler with our global handler
    { provide: ErrorHandler, useClass: GlobalErrorHandler },
  ],
};
```

Form error handling \u2014 combining form state with HTTP errors

```
// register.component.ts
@Component({
  selector: 'app-register',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <input formControlName="email" type="email" />
      @if (email.invalid && email.touched) {
        <small>
          @if (email.errors?.['required']) { Email is required }
          @if (email.errors?.['email'])    { Enter a valid email }
        </small>
      }

      <input formControlName="password" type="password" />

      <!-- HTTP-level error from the API \u2014 separate from form validation -->
      @if (submitError()) {
        <div class="api-error">{{ submitError() }}</div>
      }

      <button type="submit" [disabled]="form.invalid || submitting()">
        {{ submitting() ? 'Registering\u2026' : 'Register' }}
      </button>
    </form>
  `,
})
export class RegisterComponent {
  private fb          = inject(FormBuilder);
  private authService = inject(AuthService);
  private router      = inject(Router);
  private destroyRef  = inject(DestroyRef);

  form = this.fb.group({
    email:    ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(8)]],
  });

  submitting  = signal(false);
  submitError = signal<string | null>(null);

  get email() { return this.form.get('email')!; }

  onSubmit(): void {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }

    this.submitting.set(true);
```

```
    this.submitError.set(null);

    this.authService.register(this.form.value)
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next:  ()    => this.router.navigate(['/dashboard']),
        error: err   => {
          // HTTP error from service \u2014 show above the submit button
          this.submitError.set(err.message);
          this.submitting.set(false);
        },
      });
  }
}
```

`retry` operator \u2014 automatic HTTP retry for transient failures

```
// In a service method \u2014 retry up to 2 times on server errors
import { retry, timer } from 'rxjs';

getUsers(): Observable<User[]> {
  return this.http.get<User[]>('/api/users').pipe(
    retry({
      count: 2,
      delay: (error, retryCount) => {
        // Only retry on 5xx errors \u2014 not 4xx (client errors shouldn't be retried)
        if (error.status >= 500) {
          return timer(retryCount * 1000); // wait 1s, then 2s
        }
        throw error; // don't retry 4xx \u2014 rethrow immediately
      }
    }),
    catchError(this.handleError)
  );
}
```

Section 4 \u2014 Before & After Refactor

\u274c Before \u2014 error handling scattered and incomplete

```
// WRONG: error handling in three different bad ways simultaneously

@Component({ ... })
export class BadComponent implements OnInit {
  private http = inject(HttpClient); // \u274c HttpClient in component

  users: any[] = []; // \u274c untyped

  ngOnInit() {
    // \u274c No error callback \u2014 Observable error goes unhandled
    // \u274c No loading state \u2014 blank screen during fetch
    // \u274c No retry mechanism
    this.http.get('https://api.example.com/users').subscribe((data: any) => {
      this.users = data;
    });
  }
```

```
  onSubmit() {
    // \u274c No validity check
    // \u274c No HTTP error handling on submit
    this.http.post('/api/users', this.formData).subscribe();
  }
}
```

What breaks: HTTP errors crash silently \u2014 the Observable dies, the component gets no notification, the user sees a blank screen or stale data forever. No loading state means no feedback during the fetch. No retry means a single transient server hiccup breaks the feature permanently until page reload.

\u2705 After \u2014 layered error handling at every level

```
@Component({ ... })
export class GoodComponent {
  private userService = inject(UserService); // \u2705 service owns HTTP
  private destroyRef  = inject(DestroyRef);

  users       = signal<User[]>([]);
  loading     = signal(false);
  error       = signal<string | null>(null);
  submitting  = signal(false);
  submitError = signal<string | null>(null);

  constructor() { this.loadUsers(); }

  loadUsers(): void {
    this.loading.set(true);
    this.error.set(null);

    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next:  users => { this.users.set(users);       this.loading.set(false); },
        error: err   => { this.error.set(err.message); this.loading.set(false); },
      });
  }
}
```

Why this matters: Each layer handles what it's responsible for. The service translates HTTP status codes to readable messages. The component manages loading/error display state. The interceptor handles 401 globally. The global handler catches anything that escapes. No single layer is doing all the work.

Section 5 \u2014 Common Mistakes

Mistake 1 \u2014 Swallowing errors in `catchError` without re-throwing

What it is: `catchError(() => EMPTY)` or `catchError(() => of(null))` in a service or interceptor \u2014 the error is caught and the Observable completes silently.

Why it happens: Feels defensive. "No error = no crash." Developers want the app to stay alive.

How to avoid: Only swallow an error when you are certain the consumer can handle a null/empty result gracefully and has been designed to expect it. Otherwise always re-throw: `catchError(err => { doSomething(); return throwError(() => err); })`.

Angular consequence: The component's `error` callback never fires. The component receives `null` or `undefined` as its data. Template bindings on `null` throw `TypeError`. The original HTTP error is invisible \u2014 you debug `Cannot read properties of null` instead of `404 Not Found`.

Mistake 2 \u2014 Not separating form validation errors from HTTP errors in the template

What it is: Using the same error display for both "email is required" (form validation) and "email already registered" (API response error).

Why it happens: Both are "errors" \u2014 grouping them feels natural. Developers often show only one error area and try to handle everything there.

How to avoid: Form validation errors belong on individual fields, gated on `.touched`. API/HTTP errors belong in a separate banner or message area, shown after submit attempt. They have different lifecycles \u2014 form errors are live, API errors are point-in-time.

Angular consequence: Form validation messages and API error messages fight over the same UI real estate. A user who fixes their email gets the API error message wiped before they see it, or sees "email is required" after a server rejection \u2014 confusing UX.

Mistake 3 \u2014 Putting all error handling in the global `ErrorHandler`

What it is: Skipping `catchError` in services and component error callbacks, relying on `GlobalErrorHandler.handleError()` to catch everything.

Why it happens: "One place to handle all errors" sounds clean. The global handler feels like a safety net.

How to avoid: The global handler is the last resort, not the primary mechanism. HTTP errors caught by the global handler have lost all context \u2014 you don't know which component triggered them, what state to update, or what message to show the user.

Angular consequence: All HTTP errors show the same generic error page or message. Users can't retry, can't see field-specific feedback, and lose form state. The app technically doesn't crash but the UX is broken.

Mistake 4 \u2014 Not providing a retry path in the component

What it is: Showing an error message but no button or mechanism to retry the failed operation.

Why it happens: The happy path is built first. Error states are added as an afterthought without thinking about recovery.

How to avoid: Every error state in a component should have a clear recovery action \u2014 a Retry button that re-calls `loadData()`, a link to a different page, or instructions for the user.

Angular consequence: Users who hit a transient error (network blip, server restart) have to manually reload the entire page to try again \u2014 losing all navigation state and form progress.

Section 6 \u2014 Do's and Don'ts

\u2705 DO	\u274c DON'T
Handle errors at the lowest layer with enough context	Handle everything in the global `ErrorHandler` \u2014 it loses all component context
Always `throwError(() => err)` after handling in interceptors	Swallow errors with `EMPTY` or `of(null)` in interceptors \u2014 breaks downstream handlers
Separate form validation error display from HTTP error display	Mix form field errors and API errors in the same UI element
Provide a retry mechanism in every component error state	Show an error message with no recovery path
Clear the previous error signal before every new request	Leave stale errors on screen during a retry
Use `retry({ count: 2, delay: ... })` for idempotent GET requests	Retry POST/PUT/DELETE automatically \u2014 it can cause duplicate operations
Use signal-based `error` and `loading` state in components	Use plain mutable properties without signals \u2014 bypasses OnPush change detection
Use `status === 0` to detect network errors (request never left the browser)	Assume all errors reached the server \u2014 `status === 0` means no network

---

Section 7 \u2014 Interview Prep

Question 1

"Describe the layers of error handling in a production Angular app and what belongs at each layer."

Strong answer covers:

- Four layers: component (user-facing state), service (`catchError` translation), interceptor (cross-cutting HTTP concerns), global `ErrorHandler` (uncaught exceptions)
- Component: manages `loading`/`error` signals, shows retry buttons, field-level validation messages
- Service: translates `HttpErrorResponse` status codes to readable domain messages via `handleError`

- Interceptor: handles 401 redirect, 503 global toast, logging \u2014 concerns that apply to every request
- Global `ErrorHandler`: last resort for uncaught exceptions \u2014 logs to monitoring, optionally navigates to error page
- Key principle: handle at the lowest layer with enough context; don't try to centralise everything

Weak answer misses: The distinction between interceptor and global handler \u2014 candidates often conflate them, not realising interceptors only handle HTTP errors while `ErrorHandler` catches all unhandled JavaScript errors.

Level: Mid\u2013Senior

Question 2

"What happens if you return `EMPTY` from a `catchError` in an HTTP interceptor instead of re-throwing the error?"

Strong answer covers:

- `EMPTY` completes the Observable without emitting a value \u2014 the error is swallowed
- The service's `catchError` never runs \u2014 it's already been "handled"
- The component's `error` callback never fires
- The component receives `undefined`/`null` as its response data
- Template bindings on that null data throw `TypeError: Cannot read properties of null`
- The original HTTP error is invisible \u2014 you end up debugging unrelated TypeErrors
- Rule: interceptors must always `throwError(() => error)` unless they are certain the consumer can handle a null/empty result

Weak answer misses: The downstream consequence \u2014 candidates often say "the error is swallowed" without explaining that this turns an HTTP error into a confusing null-data `TypeError` in the component.

Level: Mid

Question 3

"How would you handle a scenario where the same API error (e.g. 401) needs both a global redirect to login AND a per-component message for users who are mid-form?"

Strong answer covers:

- The interceptor handles the global redirect for unauthenticated navigation
- The component's `error` callback still fires (because the interceptor re-throws) and can show a contextual message: "Your session expired. Please log in again to continue."

- The form state is preserved in the component until navigation completes \u2014 the user sees the message before being redirected
- A `returnUrl` query param on the redirect ensures the user lands back after login
- The key is that both handlers fire: interceptor for redirect, component for in-context messaging \u2014 because `throwError` passes the error downstream

Weak answer misses: That both can fire simultaneously \u2014 candidates often think it's one or the other, not understanding that re-throwing from the interceptor means downstream handlers all still receive the error.

Level: Senior

Section 8 \u2014 Quick Reference Card

3-line concept definition:

Angular error handling operates across four layers \u2014 component, service, interceptor, and global handler \u2014 each responsible for errors at a specific scope. Service-level `catchError` translates HTTP errors to readable messages; interceptors handle cross-cutting concerns like 401 redirects; the global `ErrorHandler` catches unhandled exceptions. Always re-throw in interceptors so downstream handlers can still respond.

Layer decision guide:

Error type	Handle at	Tool
Field validation failed	Component template	`@if (control.invalid &amp;&amp; control.touched)`
API call failed \u2014 show per-page message	Component	`error` signal + `error` callback in `.subscribe()`
HTTP error translation (code \u2192 message)	Service	`catchError` + `throwError` in `handleError`
401 \u2014 redirect to login globally	Interceptor	`catchError` + `router.navigate()` + `throwError`
Global toast for 5xx	Interceptor	`catchError` + `toastService.show()` + `throwError`
Uncaught JavaScript exception	Global `ErrorHandler`	`implements ErrorHandler` registered in `app.config.ts`
Transient server error \u2014 retry silently	Service	`retry({ count: 2, delay: timer(1000) })`

Syntax at a glance:

Task	How	
Translate HTTP error in service	`catchError(this.handleError)` \u2192 `throwError(() => new Error(msg))`	
Component error state	`error = signal<string \	null>(null)`
Clear error before retry	`this.error.set(null)` at start of `loadData()`	
Global handler registration	`{ provide: ErrorHandler, useClass: GlobalErrorHandler }` in `app.config.ts`	
Detect network error	`error.status === 0`	
Retry on transient failure	`retry({ count: 2, delay: (err) => err.status >= 500 ? timer(1000) : throwError(() => err) })`	

The one rule that matters most:

Never swallow errors in interceptors \u2014 always `throwError(() => error)` after handling, so service-level `catchError` and component `error` callbacks can still respond with context-appropriate messages and UI updates.